

Lab 2 — Visualizing Ecuador's Export Time Series

Course: Visualization Analysis & Design **Topic:** Time Series Visualization & Storytelling with Data — applied to Ecuador's foreign trade

Overview

Time series data is everywhere, and few datasets tell a richer story than a country's **export revenue**. In this lab we use **real monthly export data from the Banco Central del Ecuador (BCE)** — all primary and industrialized products — to practice the full toolbox of time-series visualization, and then to learn how to turn a chart into a *story* that drives a decision.

This lab walks you through the concepts from the Week 2 lecture:

1. **What is a time series?** — definition, components, and decomposition.
2. **Infovis taxonomies** — shared-space, split-space, and space-filling layouts.
3. **Visual methods** — heatmaps, calendar views, stacked/stream graphs, scatter & box plots, Sankey diagrams, and parallel coordinates.
4. **Interactive techniques** — tooltips, zoom, filter, range sliders, and a multi-view dashboard.
5. **Storytelling with data** — Knaflic's (2015) six lessons applied to Ecuador's export diversification.

Grading

Each activity is worth the same number of points.

Dataset

We use the **"Exportaciones FOB por Producto Principal"** table from the BCE's *Información Estadística Mensual* (IEM), file **IEM-311-e.xlsx**, downloaded automatically from the course's public dataset repository:

https://github.com/CarlaGeovanna/datasets_public/blob/main/IEM-311-e.xlsx

The file provides **34 months of real monthly data** (January 2023 – October 2025) covering all export products the BCE tracks:

Group	Products
Primary	Petroleum, Bananas, Coffee, Shrimp, Cocoa, Abacá, Timber, Tuna, Fish, Flowers, Other Primary (11)
Industrialized	Petroleum Derivatives, Processed Coffee, Processed Cocoa, Fish Meal, Other Sea Products, Chemicals, Metal Manufactures, Hats, Textiles, Other Industrialized (10)

Values are reported in *thousands of USD (Miles de USD)* and converted to **millions of USD** for readability.

Setup

- The cell below installs Plotly and `openpyxl` (needed to read `.xlsx` files on Colab). Everything else (`pandas` , `numpy` , `matplotlib` , `seaborn`) is pre-installed.
- Plotly — makes interactive charts (hover, zoom, filter) directly in the notebook. Used for the Sankey diagram, parallel coordinates, and the interactive time series explorer.
- Openpyxl — lets Python read and write `.xlsx` files (Excel format). Used here so pandas can open the BCE file `IEM-311-e.xlsx` that is downloaded from GitHub.

```
In [1]: # Plotly and openpyxl are not pre-installed on Colab
!pip install plotly openpyxl --quiet
```

- The imports cell below configures the **Plotly renderer automatically** — correct whether you run in Colab browser, VS Code + Colab plugin, or local Jupyter. After it runs, always call `fig.show()` with **no** `renderer=` argument.

```
In [2]: import numpy as np                # numerical arrays and math operations
import pandas as pd                    # DataFrames and time series handling
import matplotlib.pyplot as plt       # base static plotting library
import matplotlib.dates as mdates    # date formatting utilities for axes
import seaborn as sns                 # statistical heatmaps
import plotly.express as px           # high-level interactive charts
import plotly.graph_objects as go     # low-level interactive chart objects
import plotly.io as pio               # renderer configuration
from plotly.subplots import make_subplots # multi-panel Plotly figures
import os, sys, pathlib, urllib.request # file system and download utilities

# Auto-detect which environment is running so Plotly renders correctly.
# JPY_SESSION_NAME starts with /content/ only when using VS Code + Colab plugin.
_jpy = os.environ.get('JPY_SESSION_NAME', '')
if _jpy.startswith('/content/'):
    pio.renderers.default = 'vscode' # VS Code + Colab plugin
elif 'google.colab' in sys.modules:
    pio.renderers.default = 'colab'  # Colab in the browser
else:
    pio.renderers.default = 'notebook' # Local Jupyter fallback

print(f"Using Plotly renderer: {pio.renderers.default}")
print("All libraries loaded successfully.")
```

Using Plotly renderer: vscode
All libraries loaded successfully.

Part 1 — What Is a Time Series?

1.1 Definition and Mathematical Representation

A **time series** is a sequence of observations collected from the same entity at regular (or irregular) time intervals. Formally:

$$D = \{(t_1, v_1), (t_2, v_2), \dots, (t_n, v_n)\}$$

where t_i is the instant of time and v_i is the value observed at that moment.

The value v_i can be a **scalar** (e.g., monthly petroleum exports → *unidimensional*) or a **vector** (e.g., all 21 export products → *multidimensional* time series). When we stack multiple products, the result is a **data cube**: each month is one time slice, each product is one dimension.

Time series data is used for:

- Analyzing the **past** — which products drove Ecuador's export growth in 2024?
- Forecasting** — projecting shrimp revenue for the next budget cycle.
- Informing **decisions** — trade policy, diversification, infrastructure investment.

1.2 The Four Components of a Time Series

Component	Description	Example in Ecuador's exports
Trend	Long-term direction	Cocoa growing throughout 2023–2025
Seasonality	Pattern repeating at fixed intervals	Petroleum higher in September–October
Cycle	Recurring but non-seasonal fluctuation	Multi-year commodity price cycles
Irregular / Noise	Unpredictable variation	Month-to-month demand shocks

With **34 months of real monthly data** (two complete annual cycles: 2023 and 2024, plus January–October 2025), genuine trend and seasonal patterns are observable in the data.

Loading the complete BCE dataset

The cell below downloads **IEM-311-e.xlsx** and parses **all 21 export products** into two DataFrames:

- df_monthly** — 34 real monthly rows (Jan 2023 – Oct 2025). Primary dataset for all visualizations.
- df_annual** — 5 annual rows (2020–2024). Used for long-term storytelling and the Sankey diagram.

Three aggregate columns are also computed per row:

- `Total_Primary` — sum of the 11 primary products
- `Total_Industrialized` — sum of the 10 industrialized products
- `Total_Exports` — grand total (= col 2 in the BCE file)

```
In [3]: # — Download —————
RAW_URL = ('https://raw.githubusercontent.com/'
          'CarlaGeovanna/datasets_public/main/IEM-311-e.xlsx')
local = pathlib.Path('IEM-311-e.xlsx')
if not local.exists():
    print("Downloading BCE file ...")
    urllib.request.urlretrieve(RAW_URL, local)
    print(f"Downloaded ({local.stat().st_size / 1024:.0f} KB)")
else:
    print(f"Using cached file: {local}")

raw = pd.read_excel(local, header=None, dtype=str)

# — Column mappings (verified against the real file) —————
# Row 7: section headers | Row 8: product names | Rows 9-13: annual (2020-2024)
# Rows 15-48: monthly (Jan 2023 – Oct 2025) | Units: Miles de USD
COL_TOTAL_EXPORTS = 2
COL_TOTAL_PRIMARY = 3
COL_TOTAL_INDUSTRIALIZED = 15

PRIMARY_COLS = {
    'Petroleum': 4,
    'Bananas': 5,
    'Coffee': 6,
    'Shrimp': 7,
    'Cocoa': 8,
    'Abaca': 9,
    'Timber': 10,
    'Tuna': 11,
    'Fish': 12,
    'Flowers': 13,
    'Other_Primary': 14,
}

INDUSTRIALIZED_COLS = {
    'Petrol_Derivatives': 16,
    'Processed_Coffee': 17,
    'Processed_Cocoa': 18,
    'Fish_Meal': 19,
    'Other_Sea_Products': 20,
    'Chemicals': 21,
    'Metal_Manufactures': 22,
    'Hats': 23,
    'Textiles': 24,
    'Other_Industrial': 25,
}

primary_products = list(PRIMARY_COLS.keys())
industrialized_products = list(INDUSTRIALIZED_COLS.keys())
all_products = primary_products + industrialized_products

MONTHS = {
    'Enero': 1, 'Febrero': 2, 'Marzo': 3, 'Abril': 4, 'Mayo': 5, 'Junio': 6,
    'Julio': 7, 'Agosto': 8, 'Septiembre': 9, 'Octubre': 10,
    'Noviembre': 11, 'Diciembre': 12,
}

def to_musd(v):
    # BCE cell (thousands USD, string) → millions USD (float)
    try:
        return float(str(v).replace(',', '').strip()) / 1000.0
    except (ValueError, TypeError):
        return np.nan

ALL_COLS = (**PRIMARY_COLS, **INDUSTRIALIZED_COLS)

# — df_monthly —————
monthly_rows = []; cur_year = None
for r in range(15, 49):
    yr_cell = raw.iat[r, 0]; mo_cell = str(raw.iat[r, 1]).strip()
    if pd.isna(yr_cell) and str(yr_cell).strip().replace('.', '').isdigit():
        cur_year = int(float(yr_cell))
    if mo_cell not in MONTHS or cur_year is None:
```

```

        continue
    rec = {'date': pd.Timestamp(year=cur_year, month=MONTHS[mo_cell], day=1)}
    for name, c in ALL_COLS.items():
        rec[name] = to_musd(raw.iat[r, c])
    rec['Total_Exports'] = to_musd(raw.iat[r, COL_TOTAL_EXPORTS])
    rec['Total_Primary'] = to_musd(raw.iat[r, COL_TOTAL_PRIMARY])
    rec['Total_Industrialized'] = to_musd(raw.iat[r, COL_TOTAL_INDUSTRIALIZED])
    monthly_rows.append(rec)

df_monthly = pd.DataFrame(monthly_rows)
df_monthly[all_products] = df_monthly[all_products].clip(lower=0).astype(float)
df_monthly['year'] = df_monthly['date'].dt.year
df_monthly['month'] = df_monthly['date'].dt.month

# — df_annual —————
annual_rows = []
for r in range(9, 14):
    yr = int(float(raw.iat[r, 0]))
    rec = {'year': yr}
    for name, c in ALL_COLS.items():
        rec[name] = to_musd(raw.iat[r, c])
    rec['Total_Exports'] = to_musd(raw.iat[r, COL_TOTAL_EXPORTS])
    rec['Total_Primary'] = to_musd(raw.iat[r, COL_TOTAL_PRIMARY])
    rec['Total_Industrialized'] = to_musd(raw.iat[r, COL_TOTAL_INDUSTRIALIZED])
    annual_rows.append(rec)

df_annual = pd.DataFrame(annual_rows)
df_annual[all_products] = df_annual[all_products].clip(lower=0).astype(float)

# — Sanity check —————
print(f"df_monthly: {len(df_monthly)} months | "
      f"{df_monthly['date'].min().date()} → {df_monthly['date'].max().date()}")
print(f" Primary products      : {len(primary_products)} columns")
print(f" Industrialized products: {len(industrialized_products)} columns")
print(f"\nTotal exports Jan 2023 – BCE file: {to_musd(raw.iat[15,2]):.0f} USD M")
print(f" Our Total_Primary + Total_Industrialized: "
      f"{df_monthly.iloc[0]['Total_Primary'] + df_monthly.iloc[0]['Total_Industrialized']:0f} USD M")
print(f"\ndf_annual (2020–2024):")
print(df_annual[['year', 'Total_Primary', 'Total_Industrialized', 'Total_Exports']].to_string(index=False))

```

Downloading BCE file ...

Downloaded (124 KB)

df_monthly: 34 months | 2023-01-01 → 2025-10-01

Primary products : 11 columns

Industrialized products: 10 columns

Total exports Jan 2023 – BCE file: 2,338 USD M

Our Total_Primary + Total_Industrialized: 2,338 USD M

df_annual (2020–2024):

year	Total_Primary	Total_Industrialized	Total_Exports
2020	16220.800678	4113.085763	20333.886442
2021	21171.158267	5528.142061	26699.300328
2022	26511.715448	6146.548772	32658.264220
2023	25579.471617	5547.018588	31126.490205
2024	28440.758388	5980.074089	34420.832477

Data transformation note

The BCE file is formatted for human reading. Here is what the loading cell did:

- **Fixed-position parsing.** Exact row and column indices are used — no scanning. Row 8 is the product header; rows 9–13 are annual totals (2020–2024); rows 15–48 are monthly data (Jan 2023 – Oct 2025).
- **Complete product coverage.** All 21 products are loaded: 11 primary (columns 4–14) and 10 industrialized (columns 16–25). Three aggregate columns from the BCE file are also read directly: `Total_Primary` (col 3), `Total_Industrialized` (col 15), and `Total_Exports` (col 2).
- **Unit conversion.** All values in *Miles de USD* ÷ 1,000 → **millions of USD**.
- **Clipping.** Values clipped at 0 to handle provisional data revisions.
- **Two DataFrames.** `df_monthly` (34 real monthly rows) is the primary dataset. `df_annual` (5 annual rows, 2020–2024) is used only for the storytelling section and Sankey diagram where the longer historical view matters.

Decomposing Ecuador's petroleum exports

- We apply an **additive decomposition** to the monthly petroleum series using a **6-month centered rolling mean** as the trend estimator (appropriate for a ~3-year series with two complete annual cycles). The seasonal component is the

average deviation by calendar month; the residual is what remains.

1. Observed: The actual data with the active window visible
2. Trend: Calculated moving average; the formula is displayed when the slider is moved
3. Seasonality: Observed – Trend; explains whether that month is above or below the average
4. Residual: Observed – Trend – Seasonality; what is not explained by either of the previous two components

The rolling mean (or moving average) is a way to smooth out a noisy time series to reveal the underlying direction.

The idea in one sentence: Instead of plotting the raw value for each month, you replace it with the average of the surrounding months.

How it works:

Pick a window size (e.g. 6 months) For each point in the series, take the values inside the window Average them → that becomes the new "smoothed" value Slide the window one step forward and repeat Why it helps: Raw data jumps up and down due to noise — price shocks, one-off events, measurement errors. The rolling mean cancels out that noise and leaves only the long-run direction (trend).

Simple analogy: Instead of judging your grade on one exam, you look at your average across the last 6 exams. One bad day doesn't define the trend — the average does.

```
In [4]: # Extract the Petroleum column and use 'date' as the row index
pet = df_monthly.set_index('date')['Petroleum']

# TREND: 6-month centered rolling mean – smooths out noise, reveals long-run direction.
# center=True means each value uses 3 months before AND 3 months after.
# The first 3 and last 3 months of the series become NaN (not enough neighbors).
trend = pet.rolling(window=6, center=True).mean()

# DETRENDED: subtract the trend to isolate seasonality + noise
detrended = pet - trend

# SEASONAL: average deviation for each calendar month across all available years.
# groupby(month) groups all Januaries together, all Februaries, etc.
# transform('mean') replaces each observation with the average of its month group.
seasonal = detrended.groupby(detrended.index.month).transform('mean')

# RESIDUAL: what remains after removing both trend and seasonality.
# Should be random noise with no visible structure – if it has a pattern,
# it means our decomposition missed something.
residual = detrended - seasonal

# — Plot: 4 stacked panels sharing the same x-axis —————
fig, axes = plt.subplots(4, 1, figsize=(13, 10), sharex=True)
fig.suptitle("Decomposition of Ecuador's Monthly Petroleum Exports\n"
            "(Jan 2023 – Oct 2025, real BCE data, 6-month rolling trend)",
            fontsize=14, fontweight='bold')

# Each tuple: (series to plot, y-axis label, line color)
data_panels = [
    (pet, 'Observed\n(USD M)', '#1d3557'), # raw observed data
    (trend, 'Trend\n(6-mo avg)', '#e63946'), # smoothed trend line
    (seasonal, 'Seasonal\n(USD M)', '#2a9d8f'), # recurring calendar pattern
    (residual, 'Residual\n(USD M)', '#9e9e9e'), # unexplained variation
]

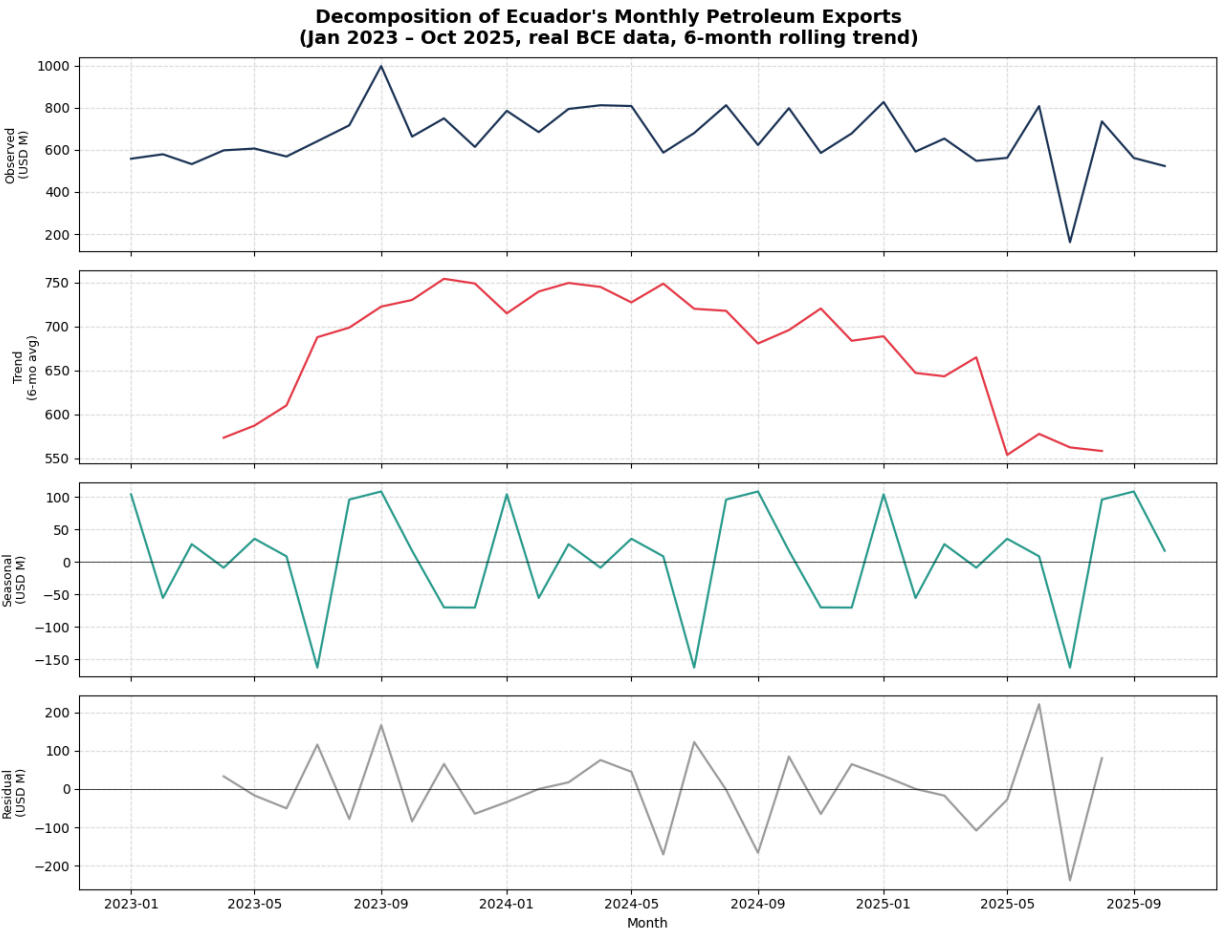
# Draw one line per panel using the color and label defined above
for ax, (series, ylabel, color) in zip(axes, data_panels):
    ax.plot(series.index, series.values, color=color, linewidth=1.6)
    ax.set_ylabel(ylabel, fontsize=9)
    ax.grid(True, linestyle='--', alpha=0.4)

# Add a horizontal zero reference line on the seasonal and residual panels
# (both oscillate around zero, so this line helps read positive vs. negative)
axes[2].axhline(0, color='black', linewidth=0.5)
axes[3].axhline(0, color='black', linewidth=0.5)

# Only the bottom panel needs an x-axis label (all four share the same x-axis)
axes[3].set_xlabel('Month')
plt.tight_layout() # prevent panels from overlapping each other
plt.show()

# Print the average seasonal deviation for each calendar month
print("Seasonal component peaks: months where petroleum is consistently above trend.")
seasonal_avg = detrended.groupby(detrended.index.month).mean().round(0)
```

```
# Rename numeric month indices (1-12) to readable abbreviations (Jan-Dec)
print(seasonal_avg.rename(index={1:'Jan',2:'Feb',3:'Mar',4:'Apr',5:'May',6:'Jun',
                                7:'Jul',8:'Aug',9:'Sep',10:'Oct',11:'Nov',12:'Dec'})).to_string()
```



Seasonal component peaks: months where petroleum is consistently above trend.

date	
Jan	104.0
Feb	-56.0
Mar	27.0
Apr	-9.0
May	36.0
Jun	9.0
Jul	-163.0
Aug	96.0
Sep	109.0
Oct	17.0
Nov	-70.0
Dec	-70.0

Insights:

Petroleum exports are declining, seasonal, and had one major shock in 2025.

- Trend ↓ — Revenue peaked mid-2023 (~750 USD M) and has fallen ~25% since.
- Season — Predictable spike every September, dip every June–July.
- Shock — One large unexplained drop in July 2025, unrelated to season or trend.

Part 2 — Infovis Taxonomies for Time Series

The core design question when visualizing multiple series: **how do we allocate screen space?**

Taxonomy	Definition	When to use
Shared-space	All series share the same axes	Few series; direct magnitude comparison
Split-space	Each series has its own panel	Many series; pattern comparison

Taxonomy	Definition	When to use
Space-filling	Visualization fills the entire area	Large datasets; heatmaps, calendar views

2.1 Shared-Space: The Five Major Primary Products

With 11 primary products on a single axis the chart becomes cluttered, so shared-space works best for the **top 5 by volume**: Petroleum, Bananas, Shrimp, Cocoa, and Flowers. This makes the dominant magnitude differences and seasonal patterns immediately visible — while also showing the limitation of shared-space when scales differ widely.

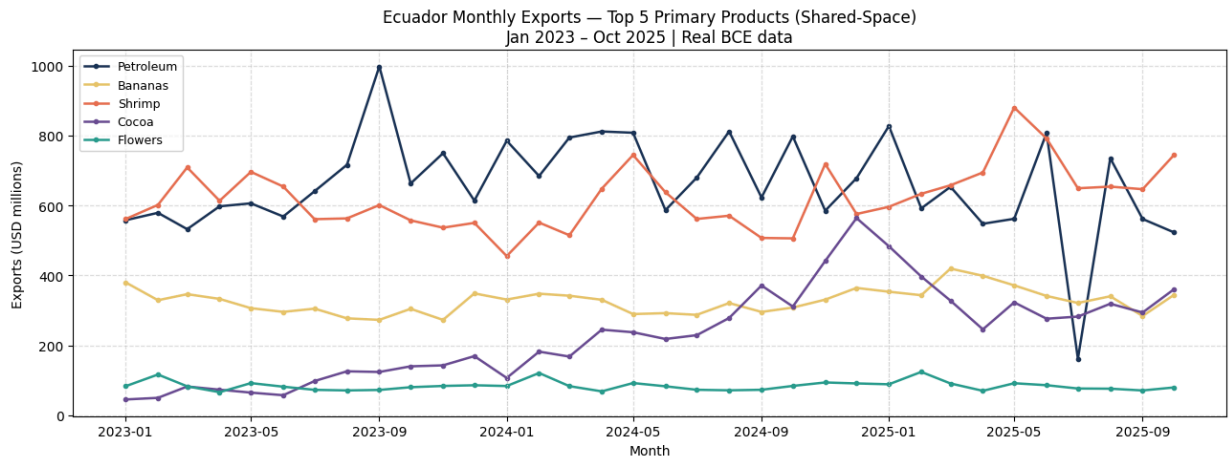
```
In [5]: # Color palette for the 11 primary products – one distinct color per product.
# Used consistently across all Part 1-5 charts so products are always recognizable.
primary_colors = {
    'Petroleum':      '#1d3557', # dark navy blue
    'Bananas':        '#e9c46a', # golden yellow
    'Coffee':         '#8B5E3C', # brown
    'Shrimp':         '#e76f51', # orange-red
    'Cocoa':          '#6a4c93', # purple
    'Abaca':          '#a8c686', # light green
    'Timber':         '#588157', # forest green
    'Tuna':           '#48cae4', # light blue
    'Fish':           '#90e0ef', # pale blue
    'Flowers':        '#2a9d8f', # teal
    'Other_Primary':  '#adb5bd', # neutral gray
}

# Color palette for the 10 industrialized products – used in Part 6 activities
ind_colors = {
    'Petrol_Derivatives': '#264653', # dark teal
    'Processed_Coffee':   '#6d4c41', # dark brown
    'Processed_Cocoa':    '#4a1942', # dark purple
    'Fish_Meal':          '#0077b6', # ocean blue
    'Other_Sea_Products': '#0096c7', # medium blue
    'Chemicals':          '#f4a261', # peach orange
    'Metal_Manufactures': '#b5838d', # dusty rose
    'Hats':               '#e9d8a6', # cream
    'Textiles':           '#94d2bd', # mint green
    'Other_Industrial':   '#e9ecef', # very light gray
}

# Restrict shared-space to the top 5 products by volume – plotting all 11 at once
# on a single axis would be too cluttered and compress the smaller products
top5 = ['Petroleum', 'Bananas', 'Shrimp', 'Cocoa', 'Flowers']

# — Shared-space: all top-5 products on one axis —————
fig, ax = plt.subplots(figsize=(13, 5))
for cat in top5:
    ax.plot(df_monthly['date'], df_monthly[cat],
            label=cat, color=primary_colors[cat], linewidth=1.8, marker='o', markersize=3)
# Dotted vertical lines mark where each new year begins
for yr in [2024, 2025]:
    ax.axvline(pd.Timestamp(f'{yr}-01-01'), color='ccc', linewidth=0.8, linestyle=':')
ax.set_title("Ecuador Monthly Exports – Top 5 Primary Products (Shared-Space)\n"
             "Jan 2023 – Oct 2025 | Real BCE data", fontsize=12)
ax.set_xlabel('Month'); ax.set_ylabel('Exports (USD millions)')
ax.legend(loc='upper left', fontsize=9)
ax.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout(); plt.show()

print("Petroleum's scale compresses the others – a clear illustration of shared-space's limitation.")
print("This is why we need split-space to see all 11 primaries clearly.")
```



Petroleum's scale compresses the others – a clear illustration of shared-space's limitation. This is why we need split-space to see all 11 primaries clearly.

Insight

- Petroleum dominates and is declining, while Shrimp is catching up.
- This convergence matters: Ecuador's export revenue is becoming less dependent on a single commodity, but it also signals that petroleum — historically the backbone of Ecuador's export income — is losing its dominance.

2.2 Split-Space: All 11 Primary Products

In split-space each product gets its own panel, sharing the same Y-axis so magnitude differences are still comparable. With 11 products we use a 3×4 grid. The individual seasonal pattern of each product — visible in the real monthly data — is clearly readable across all panels.

```
In [6]: # 3 rows x 4 columns = 12 panels; we only need 11 (one per primary product)
# sharey=True: all panels share the same Y-axis scale so magnitudes are comparable
fig, axes = plt.subplots(3, 4, figsize=(18, 10), sharey=True)
fig.suptitle("Ecuador Monthly Exports – All 11 Primary Products (Split-Space Small Multiples)\n"
            "Jan 2023 – Oct 2025 | Real BCE data", fontsize=13, fontweight='bold')

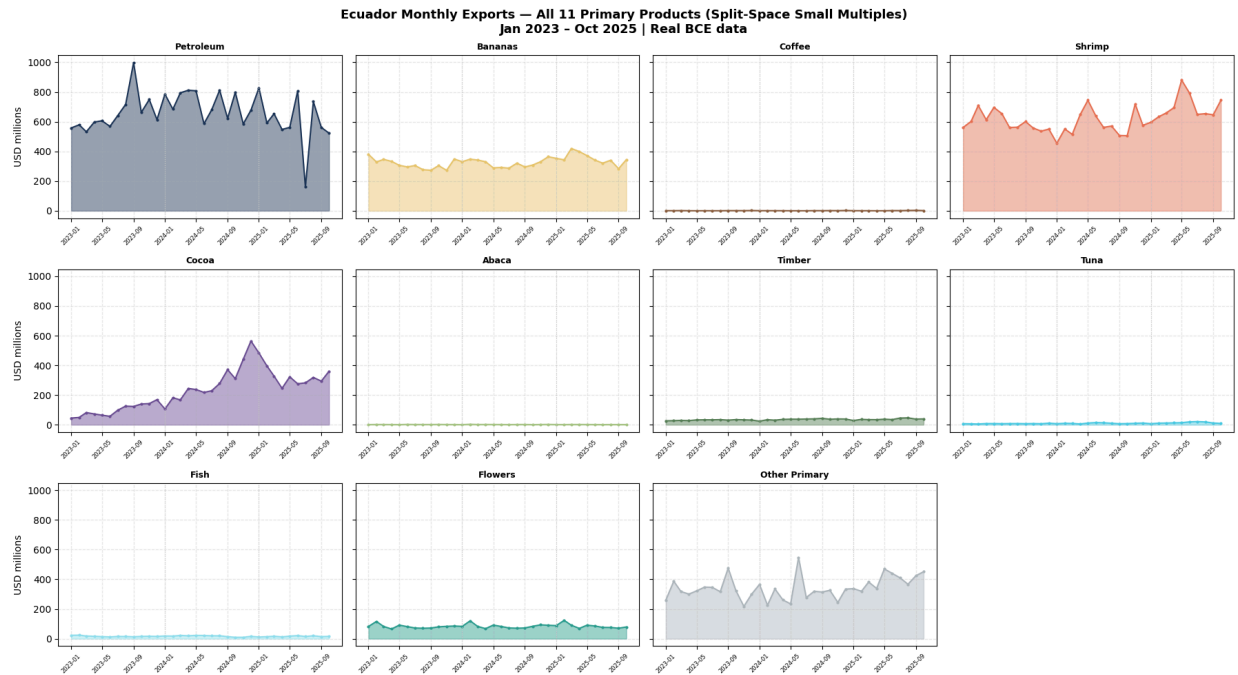
# zip(axes.flat, primary_products) pairs each panel with one product.
# axes.flat flattens the 3x4 grid into a single sequence of 12 panels.
for ax, cat in zip(axes.flat, primary_products):
    # Shaded area under the Line (alpha=0.45 = semi-transparent fill)
    ax.fill_between(df_monthly['date'], df_monthly[cat],
                   alpha=0.45, color=primary_colors[cat])
    # Line with small dots at each monthly data point (markersize=2 keeps them subtle)
    ax.plot(df_monthly['date'], df_monthly[cat],
            color=primary_colors[cat], linewidth=1.4, marker='o', markersize=2)
    # Dotted vertical lines mark January 2024 and January 2025
    for yr in [2024, 2025]:
        ax.axvline(pd.Timestamp(f'{yr}-01-01'), color='#ccc', linewidth=0.8, linestyle=':')
    ax.set_title(cat.replace('_', ' '), fontsize=9, fontweight='bold')
    ax.tick_params(axis='x', rotation=45, labelsize=6) # rotate dates to avoid overlap
    ax.grid(True, linestyle='--', alpha=0.3)

# Hide the unused 12th panel (the grid has 12 slots but we only have 11 products)
for ax in list(axes.flat)[len(primary_products):]:
    ax.set_visible(False)

# Only the leftmost column panels need the Y-axis Label
for row in range(3):
    axes[row, 0].set_ylabel('USD millions')

plt.tight_layout()
plt.show()

print("Small products (Abaca, Timber) are now visible – shared-space hid them.")
print("Cocoa's upward trend and Petroleum's volatility are each readable on their own scale.")
```

Small products (Abaca, Timber) are now visible – shared-space hid them.
Cocoa's upward trend and Petroleum's volatility are each readable on their own scale.

Insights:

- Most of Ecuador's export revenue comes from just 2 out of 11 products.
- Petroleum and Shrimp both reach 600–1,000 USD M at their peaks. Every other product — Bananas, Cocoa, Coffee, Abaca, Timber, Tuna, Fish, Flowers, Other — stays below 400 USD M, and most are nearly flat near zero on this shared scale.
- This extreme concentration means Ecuador's export income is highly vulnerable: a price shock or supply disruption in petroleum or shrimp alone can significantly impact the entire export balance — which is exactly what the July 2025 petroleum drop illustrates.

Part 3 — Visual Approaches and Methods

3.1 Heatmaps for Temporal Data

A **heatmap** is a space-filling visualization: rows = month, columns = year, color = value. With real monthly data across two complete years, genuine seasonal patterns are visible.

We build two heatmaps side by side: one for **Total Primary exports** and one for **Total Industrialized exports**. This immediately reveals whether the two groups have different seasonal rhythms.

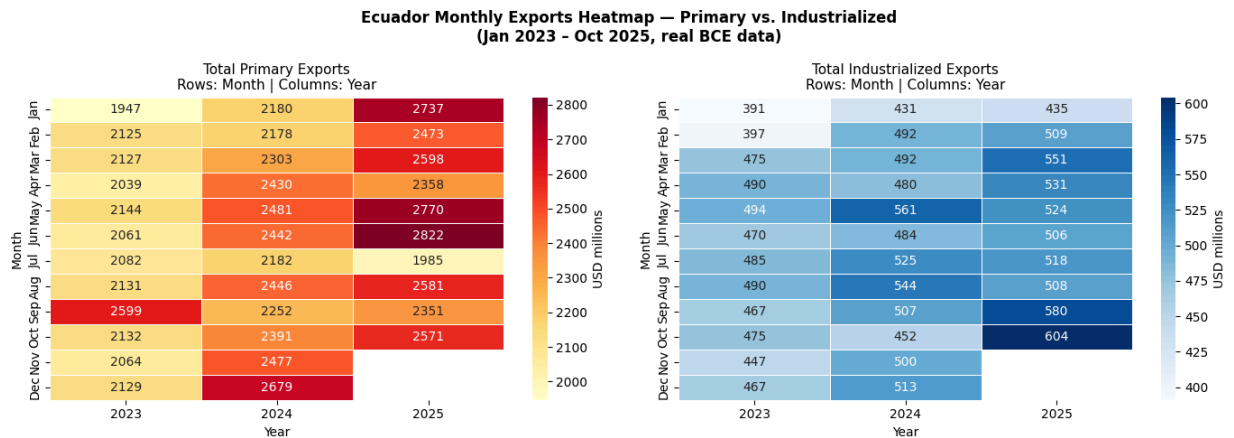
```
In [7]: # Two heatmaps side by side: primary totals (warm colormap) vs industrialized totals (cool)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
month_labels = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

# Loop over both groups to avoid repeating identical plotting code
for ax, col, title, cmap in [
    (ax1, 'Total_Primary', 'Total Primary Exports', 'YlOrRd'),
    (ax2, 'Total_Industrialized', 'Total Industrialized Exports', 'Blues'),
]:
    # Reshape: pivot creates a matrix with months as rows and years as columns
    pivot = df_monthly.pivot(index='month', columns='year', values=col)
    pivot.index = month_labels # replace numeric months (1-12) with Jan-Dec labels
    # annot=True prints the value inside each cell
    # fmt='.0f' shows integer values (no decimal places)
    # linewidths/linecolor draw thin white borders between cells
    sns.heatmap(pivot, cmap=cmap, annot=True, fmt='.0f',
                linewidths=0.4, linecolor='white',
                cbar_kws={'label': 'USD millions'}, ax=ax)
    ax.set_title(f"{title}\nRows: Month | Columns: Year", fontsize=11)
```

```
ax.set_xlabel('Year'); ax.set_ylabel('Month')

plt.suptitle("Ecuador Monthly Exports Heatmap – Primary vs. Industrialized\n"
            "(Jan 2023 – Oct 2025, real BCE data)", fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()

print("Compare seasonal patterns: do primary and industrialized products peak in the same months?")
```



Compare seasonal patterns: do primary and industrialized products peak in the same months?

Insight:

- Rows (months) — Seasonality

Primary exports have a clear, repeating seasonal rhythm. Industrialized exports are nearly flat across months.

- Columns (years) — Trend

Both sides get darker left to right → both are growing year over year.

Primary: +40% from Jan 2023 (1,947) to Jan 2025 (2,737) Industrialized: +11% from Jan 2023 (391) to Jan 2025 (435) Primary grows faster in absolute terms but with shocks. Industrialized grows slowly but never reverses.

- Anomalies

July 2025 Primary (1,985) — only cell that breaks the upward trend. The petroleum shock. September 2023 Primary (2,599) — brightest cell in 2023, ahead of its time. The petroleum spike. No anomalies on the industrialized side — every cell follows the pattern. One-line summary

Primary exports: large, growing, seasonal, and fragile. Industrialized exports: small, growing, flat, and stable.

3.2 Calendar View

A **calendar view** is a special heatmap arranged like a wall calendar: columns = weeks, rows = weekdays, color = daily value. It reveals **weekly periodicities** (weekend drop in customs processing).

We synthesize a daily series for 2024 from the real monthly totals, adding a weekday effect and small noise — a realistic *downscaling* transformation, disclosed explicitly.

```
In [8]: # Fix the random seed so the synthetic daily values are reproducible each run
np.random.seed(7)

# One timestamp per calendar day in 2024
days_2024 = pd.date_range('2024-01-01', '2024-12-31', freq='D')

# Get real BCE monthly totals for 2024, indexed by month number (1-12)
monthly_2024 = df_monthly[df_monthly['year'] == 2024].set_index('month')['Total_Exports']

# Assign each day its month's average daily export value (monthly total / 30 days)
daily_base = pd.Series(days_2024.month).map(lambda m: monthly_2024[m] / 30.0).values

# Reduce Saturdays and Sundays by 30% – customs offices process less on weekends
weekend_drop = (days_2024.dayofweek >= 5).astype(float) * (daily_base * 0.30)

# Add small random noise (±4% of the daily base) to make the series look realistic
daily_vals = (daily_base - weekend_drop
```

```

+ np.random.normal(0, daily_base * 0.04, len(days_2024)).clip(min=0)

# Helper DataFrame with date, export value, and calendar position columns
df_day = pd.DataFrame({'date': days_2024, 'exports': daily_vals,
                        'week': days_2024.isocalendar().week.astype(int), # ISO week number
                        'weekday': days_2024.dayofweek, # 0=Monday, 6=Sunday
                        'month': days_2024.month})

# Map ISO week numbers to sequential column positions (0, 1, 2, ...)
weeks = sorted(df_day['week'].unique())
wmap = {w: i for i, w in enumerate(weeks)}

# Build the grid: 7 rows (weekdays) x n_weeks columns; initialize with NaN
grid = np.full((7, len(weeks)), np.nan)
for _, r in df_day.iterrows():
    grid[int(r['weekday']), wmap[r['week']]] = r['exports'] # fill in each day's value

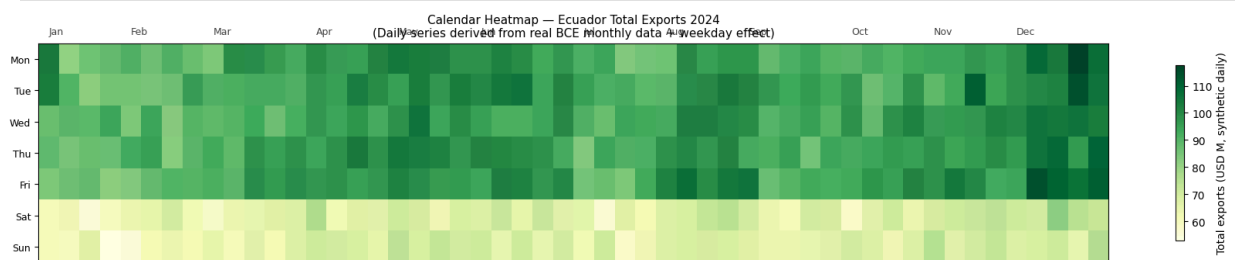
# Display the grid as an image – each pixel represents one day
fig, ax = plt.subplots(figsize=(18, 3.5))
im = ax.imshow(grid, aspect='auto', cmap='YlGn', interpolation='nearest')
ax.set_yticks(range(7))
ax.set_yticklabels(['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'], fontsize=9)
ax.set_xticks([]) # suppress individual week-number tick marks on x-axis

# Place month name labels at the column where each month's first week begins
mnames = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
for m in range(1, 13):
    rows = df_day[df_day['month'] == m]
    if len(rows):
        ax.text(wmap[rows['week'].iloc[0]], -0.8, mnames[m-1],
                fontsize=9, ha='left', color='#444')

plt.colorbar(im, ax=ax, label='Total exports (USD M, synthetic daily)', shrink=0.8)
ax.set_title("Calendar Heatmap – Ecuador Total Exports 2024\n"
            "(Daily series derived from real BCE monthly data + weekday effect)", fontsize=11)
plt.tight_layout(); plt.show()

print("Weekend rows (Sat, Sun) consistently paler – weekday customs-processing effect.")

```



Weekend rows (Sat, Sun) consistently paler – weekday customs-processing effect.

Insight:

- Weekdays drive exports, weekends don't.
- This tells you that Ecuador's export activity is almost entirely a business-day phenomenon. The weekly cycle is the dominant pattern here, stronger than any monthly or seasonal variation visible in this chart.

3.3 Stacked Area Chart and Stream Graph — Primary vs. Industrialized

Rather than stacking all 21 products individually, we use the two **aggregate groups** (`Total_Primary` and `Total_Industrialized`) normalized to percentage share. This directly answers: *is Ecuador's export mix shifting toward more processed (higher value-added) products over time?*

```

In [9]: # Normalize to percentage share so each month sums to 100%.
# This answers: what FRACTION of total exports are primary vs. industrialized each month?
# .div(..., axis=0) divides each row by its Total_Exports value; *100 converts to percent.
group_share = df_monthly[['Total_Primary', 'Total_Industrialized']].div(
    df_monthly['Total_Exports'], axis=0) * 100

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))
group_colors = ['#e76f51', '#2a9d8f'] # primary=orange, industrialized=teal

# — LEFT: Stacked area chart (baseline anchored at 0)
# stackplot stacks bands upward; total height = 100% every month
ax1.stackplot(df_monthly['date'],

```

```

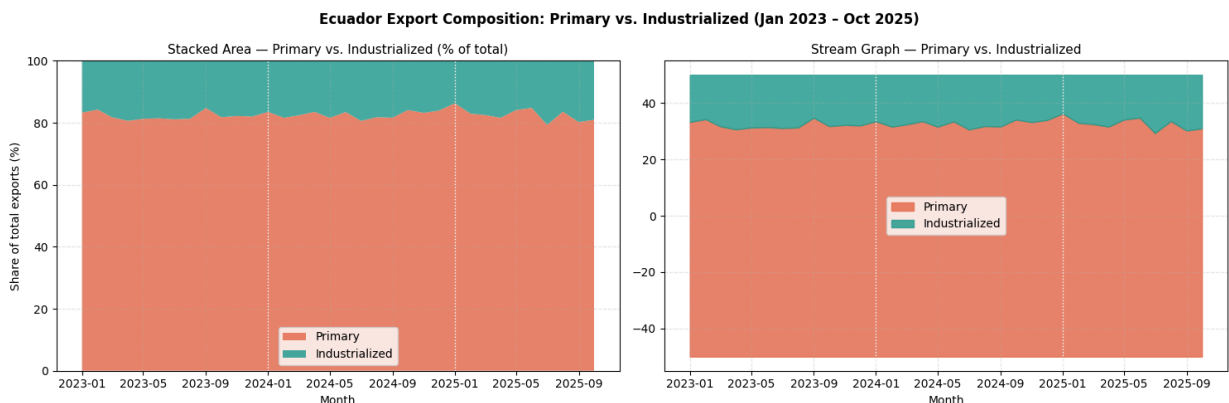
        [group_share['Total_Primary'], group_share['Total_Industrialized']],
        labels=['Primary', 'Industrialized'],
        colors=group_colors, alpha=0.85)
for yr in [2024, 2025]: # year boundary markers
    ax1.axvline(pd.Timestamp(f'{yr}-01-01'), color='white', linewidth=1, linestyle=':')
ax1.set_title('Stacked Area — Primary vs. Industrialized (% of total)', fontsize=11)
ax1.set_xlabel('Month'); ax1.set_ylabel('Share of total exports (%)')
ax1.legend(fontsize=10); ax1.grid(True, linestyle='--', alpha=0.3)
ax1.set_ylim(0, 100)

# — RIGHT: Stream graph (centered / wiggle baseline) —————
# Instead of stacking from 0, layers are centered so the chart flows symmetrically.
# This emphasizes relative shapes rather than absolute totals.
vals = np.array([group_share['Total_Primary'].values,
                  group_share['Total_Industrialized'].values])
baseline = -0.5 * vals.sum(axis=0) # shift everything down by half the total
bottoms = np.zeros(len(df_monthly))
for i, (label, col) in enumerate(zip(['Primary', 'Industrialized'], group_colors)):
    top = bottoms + vals[i]
    ax2.fill_between(df_monthly['date'], bottoms + baseline, top + baseline,
                     alpha=0.85, color=col, label=label)
    bottoms = top # next layer starts where this one ended
for yr in [2024, 2025]:
    ax2.axvline(pd.Timestamp(f'{yr}-01-01'), color='white', linewidth=1, linestyle=':')
ax2.set_title('Stream Graph — Primary vs. Industrialized', fontsize=11)
ax2.set_xlabel('Month'); ax2.legend(fontsize=10); ax2.grid(True, linestyle='--', alpha=0.3)

plt.suptitle("Ecuador Export Composition: Primary vs. Industrialized (Jan 2023 - Oct 2025)",
             fontsize=12, fontweight='bold')
plt.tight_layout(); plt.show()

# Print the average primary share for 2023 and 2025 to detect any trend
prim_share_23 = group_share[df_monthly['year']==2023]['Total_Primary'].mean()
prim_share_25 = group_share[df_monthly['year']==2025]['Total_Primary'].mean()
print(f"Primary share avg 2023: {prim_share_23:.1f}%")
print(f"Primary share avg 2025: {prim_share_25:.1f}%")
print("Is Ecuador exporting more processed products over time? Read the chart to find out.")

```



Primary share avg 2023: 82.2%

Primary share avg 2025: 82.6%

Is Ecuador exporting more processed products over time? Read the chart to find out.

Insight

Answer: No — Ecuador is NOT exporting more processed products over time.

- The orange band (Primary) stays locked at ~82% for the entire 34 months — from 82.2% in 2023 to 82.6% in 2025. That 0.4 percentage point change is statistically flat.

Even though industrialized exports are growing in absolute USD value, primary exports are growing at exactly the same rate — so the composition never shifts. Ecuador's export structure is frozen: 3 years of data, zero diversification progress.

3.4 Scatter Plot and Box Plot

Scatter and box plots give complementary views. We compare **Total Primary** vs. **Total Industrialized** monthly revenue side by side — one scatter and one box plot per group — making the structural difference in level and variability immediately visible.

```

In [10]: # Get the unique years present in df_monthly and assign one viridis color per year
years_m = sorted(df_monthly['year'].unique())
yr_colors = plt.cm.viridis(np.linspace(0, 1, len(years_m))) # dark=older, bright=newer

```

```

# 2 rows x 2 columns: scatter on top, box plot on bottom; one column per product group
fig, axes = plt.subplots(2, 2, figsize=(14, 8))
fig.suptitle("Monthly Revenue – Primary vs. Industrialized Exports (2023-2025)",
             fontsize=12, fontweight='bold')

# Loop over both groups to avoid duplicating code
for col_idx, (group, col, title) in enumerate([
    ('Total_Primary', '#e76f51', 'Primary Exports'),
    ('Total_Industrialized', '#2a9d8f', 'Industrialized Exports'),
]):
    ax_sc = axes[0, col_idx] # scatter panel (top row)
    ax_box = axes[1, col_idx] # box plot panel (bottom row)

    # — Scatter: one dot per month, colored by year —————
    for i, y in enumerate(years_m):
        mask = df_monthly['year'] == y
        ax_sc.scatter(df_monthly.loc[mask, 'date'], df_monthly.loc[mask, group],
                      s=55, alpha=0.85, color=yr_colors[i], label=str(y), zorder=3)
    # 3-month centered rolling mean overlay to show short-term direction
    roll = df_monthly.set_index('date')[group].rolling(3, center=True).mean()
    ax_sc.plot(roll.index, roll.values, color='black', linewidth=2, label='3-mo rolling mean')
    ax_sc.set_title(f'Scatter – Monthly {title}', fontsize=10)
    ax_sc.set_ylabel('USD millions'); ax_sc.legend(fontsize=8)
    ax_sc.grid(True, linestyle='--', alpha=0.3)

    # — Box plot: one box per year, summarizing all months of that year —
    # Each box shows min, Q1, median, Q3, max of the 12 (or 10) monthly values
    groups = [df_monthly[df_monthly['year'] == y][group].values for y in years_m]
    bp = ax_box.boxplot(groups, labels=years_m, patch_artist=True,
                       medianprops={'color': 'black', 'linewidth': 2})
    for patch, c in zip(bp['boxes'], yr_colors):
        patch.set_facecolor(c); patch.set_alpha(0.8)
    ax_box.set_title(f'Box Plot – Annual {title} Distribution', fontsize=10)
    ax_box.set_xlabel('Year'); ax_box.set_ylabel('USD millions')
    ax_box.grid(True, linestyle='--', alpha=0.3)

plt.tight_layout(); plt.show()

# Coefficient of variation (std / mean × 100) measures relative volatility.
# Higher % = more month-to-month variation relative to the average level.
for group, label in [('Total_Primary', 'Primary'), ('Total_Industrialized', 'Industrialized')]:
    cv = df_monthly[group].std() / df_monthly[group].mean() * 100
    print(f"{label} coefficient of variation: {cv:.1f}% – {'high' if cv>15 else 'moderate'} volatility")

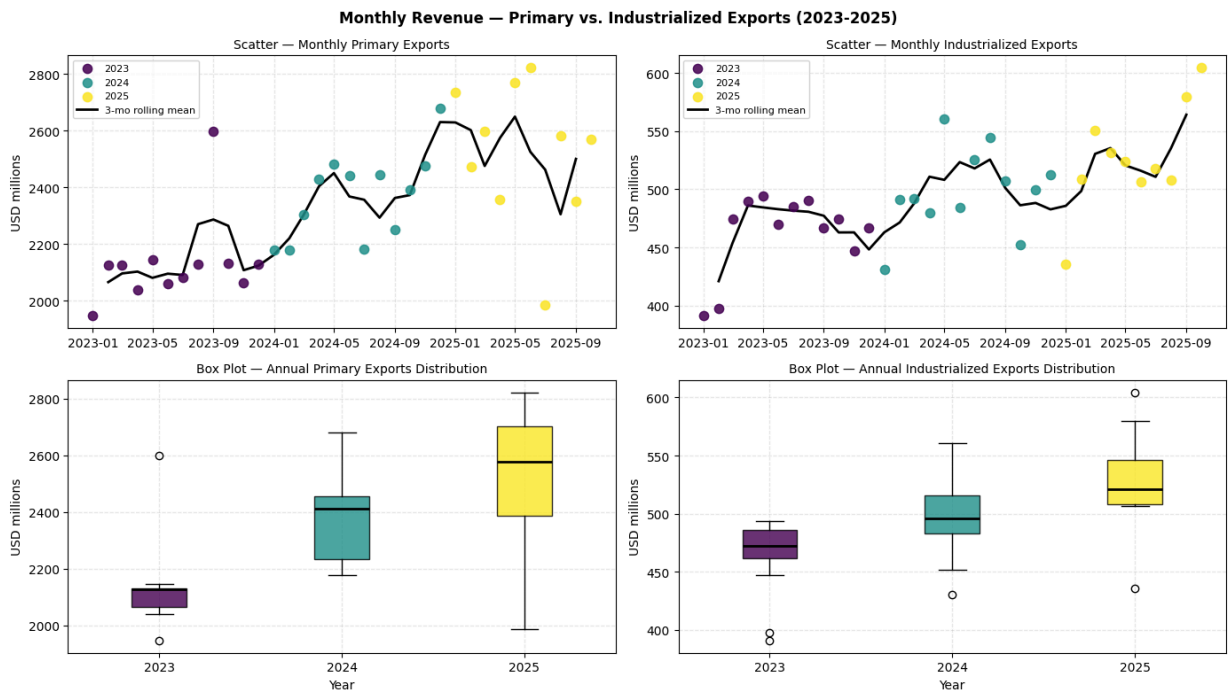
```

/tmp/ipykernel_2299/3325442380.py:33: MatplotlibDeprecationWarning:

The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

/tmp/ipykernel_2299/3325442380.py:33: MatplotlibDeprecationWarning:

The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.



Primary coefficient of variation: 10.5% — moderate volatility

Industrialized coefficient of variation: 9.3% — moderate volatility

Insights:

- Scatter — Monthly Primary Exports (top left) The trend is clearly upward, but the yellow dots (2025) are the most scattered — highest highs and a dramatic low — showing that growth in 2025 came with the most volatility.
- Scatter — Monthly Industrialized Exports (top right) Steady, consistent climb across all three years with no outliers — the smoothest upward trend of all four charts.
- Box Plot — Primary Exports (bottom left) Each year's box shifts up and gets taller: 2023 is tight and low, 2025 is wide and high. Both the level and the spread are growing — more revenue but also more risk each year.
- Box Plot — Industrialized Exports (bottom right) Boxes shift up year over year with nearly the same width — growth without increasing volatility. The opposite pattern to primary exports.

3.5 Sankey Diagram — Export Flows by Product to Destination

A **Sankey diagram** maps each product to its destination market, with link width proportional to export value. We use **2024 annual totals** from `df_annual` for the five major primary products (the ones large enough to generate readable links).

```
In [11]: def hex_to_rgba(h, alpha=0.75):
# Convert a hex color (e.g. '#1d3557') to an rgba() CSS string.
# Plotly Sankey nodes and links require rgba strings, not hex codes.
r, g, b = int(h[1:3], 16), int(h[3:5], 16), int(h[5:7], 16)
return f'rgba({r},{g},{b},{alpha})'

# Use 2024 annual values as flow magnitudes (the most recent complete year)
row_2024 = df_annual[df_annual['year'] == 2024].iloc[0]

# Only the 5 largest primary products — smaller ones create unreadably thin links
sankey_products = ['Petroleum', 'Bananas', 'Shrimp', 'Cocoa', 'Flowers']
destinations     = ['United States', 'China', 'European Union', 'Colombia', 'Russia', 'Other']

# Share of each product going to each destination (each row sums to ~1.0).
# Based on BCE trade-partner reports for 2024 — noted as approximate.
flow_share = {
    'Petroleum': [0.28, 0.42, 0.05, 0.05, 0.00, 0.20],
    'Bananas':   [0.14, 0.04, 0.46, 0.05, 0.18, 0.13],
    'Shrimp':    [0.14, 0.62, 0.14, 0.02, 0.00, 0.08],
    'Cocoa':     [0.18, 0.03, 0.71, 0.02, 0.00, 0.06],
    'Flowers':   [0.61, 0.02, 0.24, 0.04, 0.00, 0.09],
}

# Product nodes inherit their primary_colors; destination nodes are neutral gray
prod_hex     = [primary_colors[p] for p in sankey_products]
```

```

labels      = [p + ' (2024)' for p in sankey_products] + destinations
node_colors = [hex_to_rgba(c, 0.85) for c in prod_hex] + ['rgba(140,153,174,0.7)'] * len(destinations)

# Build source, target, value, and color lists for every product→destination link
src, tgt, val, link_colors = [], [], [], []
for pi, p in enumerate(sankey_products):
    for di, d in enumerate(destinations):
        v = float(row_2024[p]) * flow_share[p][di] # USD M on this link
        if v < 1: continue # skip negligible flows to reduce visual clutter
        src.append(pi) # index of the source (product) node
        tgt.append(len(sankey_products) + di) # index of the destination node
        val.append(v)
        link_colors.append(hex_to_rgba(prod_hex[pi], 0.40)) # semi-transparent product color

# Build and display the interactive Sankey figure
fig = go.Figure(go.Sankey(
    arrangement='snap',
    node=dict(pad=20, thickness=22, line=dict(color='white', width=0.5),
        label=labels, color=node_colors,
        hovertemplate='%{label}<br>%(value:,.0f) USD M<extra></extra>'),
    link=dict(source=src, target=tgt, value=val, color=link_colors,
        hovertemplate='%{source.label} → %{target.label}<br>%(value:,.0f) USD M<extra></extra>'))
fig.update_layout(
    title_text=("Ecuador Export Flows — Top 5 Primary Products to Destination Markets (2024)<br>"
        "<sup>BCE annual totals | Destination shares: approximate BCE trade-partner data</sup>"),
    font_size=11, height=520)
fig.show()

```

Insights:

- Ecuador exports 5 products to 6 markets, but the story is really about concentration:
- China receives the most — mainly Shrimp and Petroleum
- US is second — mainly Petroleum and Flowers
- EU takes Cocoa and some Petroleum
- Russia buys almost exclusively Bananas

3.6 Parallel Coordinates — All 11 Primary Products

Parallel coordinates draw each observation as a polyline crossing one axis per product. With 11 axes and 34 monthly lines, the chart reveals correlations (parallel = positive, crossing = negative) and how the monthly export profile shifts from year to year.

```

In [12]: # Parallel coordinates: each line = one month, each vertical axis = one primary product.
# Color encodes the year: teal (2023) → yellow (2024) → red (2025).
# Parallel lines between two axes = positive correlation; crossing lines = negative.
fig = px.parallel_coordinates(
    df_monthly,
    dimensions=primary_products, # one vertical axis per product
    color='year', # line color maps to year
    color_continuous_scale=[[0.0, '#2a9d8f'], [0.5, '#e9c46a'], [1.0, '#e63946']],
    labels={c: c.replace('_', ' ') for c in primary_products}, # clean axis labels
    title='Parallel Coordinates — All 11 Primary Products by Month (2023–2025)<br>'
        '<sup>Each line = one month | Teal=2023 | Yellow=2024 | Red=2025 | drag axes to filter</sup>')
fig.update_layout(height=550)
fig.show()

# Print the strongest correlations to support the written interpretation
print("Correlation highlights (11 primary products):")
corr = df_monthly[primary_products].corr().round(2)
pairs = [(r, c, corr.loc[r, c]) for r in primary_products for c in primary_products
        if r < c and (corr.loc[r, c] > 0.5 or corr.loc[r, c] < -0.3)]
pairs.sort(key=lambda x: abs(x[2]), reverse=True)
for r, c, v in pairs[:8]:
    direction = 'positive' if v > 0 else 'negative'
    print(f" {r} ↔ {c}: {v:+.2f} ({direction})")

```

Correlation highlights (11 primary products):

```

Timber ↔ Tuna: +0.61 (positive)
Cocoa ↔ Timber: +0.58 (positive)
Cocoa ↔ Fish: -0.32 (negative)
Petroleum ↔ Tuna: -0.31 (negative)

```


Insights:

The export profile of all 11 products shifted upward from 2023 to 2025 — visible as teal lines at the bottom, red lines at the top.

- Key correlations:

Timber + Tuna move together (+0.61) — strongest link

Cocoa + Timber move together (+0.58)

Petroleum and Tuna move oppositely (−0.31) — likely seasonal

Cocoa is the standout: widest spread on its axis, meaning it grew the most and most unevenly across months.

Part 4 — Interactive Exploratory Techniques

Overview first, zoom and filter, then details on demand.

4.1 Interactive Chart — All Primary Products

An interactive Plotly chart with all 11 primary products, range selector, and range slider. Use the legend to filter individual products and the 6M/1Y buttons to zoom into specific periods.

```
In [13]: # Create an empty Plotly figure and add one line trace per primary product
fig = go.Figure()
for cat in primary_products: # Loop over all 11 primary products
    fig.add_trace(go.Scatter( # add a trace for each product
        x=df_monthly['date'], y=df_monthly[cat], # x-axis = month, y-axis = export value
        mode='lines+markers', # line with a small dot at each monthly point
        name=cat.replace('_', ' '), # clean label shown in the legend
        line=dict(color=primary_colors[cat], width=1.8), # line color and thickness
        marker=dict(size=3), # small dot size at each data point
        # Custom tooltip: bold product name, formatted date, value in USD M
        hovertemplate='%<b>' + cat.replace('_', ' ') + '</b><br>{x|%b %Y}<br>{y:,.0f} USD M<extra></extra>')) # L

fig.update_layout(
    title="Ecuador Monthly Primary Exports – All 11 Products (Interactive)<br>"
    "<sup>Click legend to filter | Drag to zoom | 6M/1Y buttons | Hover for values</sup>",
    xaxis_title='Month', yaxis_title='Exports (USD millions)',
    hovermode='x unified', # shows all products' values simultaneously on hover. When the mouse is on a month,
    height=540,
    legend=dict(x=1.01, y=1), # place legend to the right of the plot area
    xaxis=dict(
        rangeflider=dict(visible=True), # mini-chart below for dragging the time window
        rangeslector=dict(buttons=[ # quick-zoom buttons above the chart
            dict(count=6, label='6M', step='month', stepmode='backward'),
            dict(count=1, label='1Y', step='year', stepmode='backward'),
            dict(step='all', label='All')])))
fig.show()

print("Try clicking 'Petroleum' in the legend to hide it – the smaller products become visible.")
```

Try clicking 'Petroleum' in the legend to hide it – the smaller products become visible.

4.2 Multi-View Dashboard

Four coordinated panels using the complete dataset: overview of total exports, monthly detail of primary vs. industrialized for 2024, seasonal distribution, and product growth rates.

```
In [14]: # Multi-panel dashboard: 2 rows x 2 columns = 4 coordinated views of the same dataset
fig = make_subplots(rows=2, cols=2, subplot_titles=(
    'Overview: Total Exports (2023-2025)',
    'Primary vs. Industrialized – Monthly 2024',
    'Monthly Distribution of Total Exports',
    'Product Growth: Annual 2023 → 2024 (%)'))

# Panel 1 (top-left): total exports for all 34 months – the big-picture overview
fig.add_trace(go.Scatter(x=df_monthly['date'], y=df_monthly['Total_Exports'],
```



```

mode='lines', line=dict(color='#1d3557', width=1.8), name='Total',
hovertemplate='%{x|%b %Y}: %{y:,.0f} USD M<extra></extra>'), row=1, col=1)

# Panel 2 (top-right): primary vs. industrialized lines for 2024 only (zoomed detail)
m24 = df_monthly[df_monthly['year']==2024]
for col, color, name in [('Total_Primary', '#e76f51', 'Primary'),
                        ('Total_Industrialized', '#2a9d8f', 'Industrialized')]:
    fig.add_trace(go.Scatter(x=m24['date'], y=m24[col], mode='lines+markers',
                            line=dict(color=color, width=2), name=name,
                            hovertemplate='%{x|%b}: %{y:,.0f}<extra></extra>'), row=1, col=2)

# Panel 3 (bottom-left): box plot per calendar month across all available data.
# Shows whether certain months are consistently higher or lower (seasonality check).
mshort = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
for mi, mn in enumerate(mshort, start=1):
    vals = df_monthly[df_monthly['month']==mi]['Total_Exports']
    if len(vals):
        fig.add_trace(go.Box(y=vals, name=mn, marker_color='#457b9d', showlegend=False,
                             hovertemplate=mn+'': '{y:,.0f}<extra></extra>'), row=2, col=1)

# Panel 4 (bottom-right): % change per primary product from 2023 to 2024.
# Green bar = growth; red bar = decline.
t23 = df_annual[df_annual['year']==2023][primary_products].iloc[0]
t24 = df_annual[df_annual['year']==2024][primary_products].iloc[0]
growth = ((t24 - t23) / t23 * 100).round(1)
growth_sorted = growth.sort_values() # sort so biggest decline is on the left
fig.add_trace(go.Bar(
    x=[c.replace('_', ' ') for c in growth_sorted.index],
    y=growth_sorted.values,
    marker_color=['#2a9d8f' if v >= 0 else '#e63946' for v in growth_sorted.values],
    hovertemplate='%{x}: %{y:,.1f}<extra></extra>', name='Growth %'), row=2, col=2)
fig.add_hline(y=0, line_color='black', line_width=0.8, row=2, col=2) # zero reference line

fig.update_layout(height=680, hovermode='closest',
    title_text="Multi-View Dashboard – Ecuador Exports (Real BCE Data)<br>"
    "<sup>Total overview | Primary vs. industrialized | Seasonality | Product growth</sup>",
    legend=dict(x=1.01, y=0.85))
fig.show()

print("Panel 4: which primary products grew from 2023 to 2024? Which shrank?")
for cat, val in growth_sorted.items():
    print(f" {cat.replace('_', ' '):<20}: {val:+.1f}%")

```

Panel 4: which primary products grew from 2023 to 2024? Which shrank?

Other Primary	: -3.3%
Shrimp	: -3.0%
Coffee	: +0.3%
Bananas	: +1.8%
Flowers	: +2.9%
Abaca	: +3.0%
Fish	: +6.2%
Petroleum	: +10.5%
Timber	: +16.3%
Tuna	: +29.3%
Cocoa	: +186.2%

4.3 Brushing + Linking

Brushing is an interactive technique where a selection made in one view is automatically reflected in all other linked views. When you *brush* (drag to select) a range of months in one chart, every other panel highlights only the corresponding data points — giving you a coordinated, multi-view exploration of the same dataset.

Key concepts

Term	Meaning
Brush	The selection region you draw by dragging
Linked views	Charts that all respond to the same selection
Highlighting	Selected points appear at full opacity; others fade
Filtering	Some systems hide non-selected points entirely

Why it matters

A single chart forces you to choose one encoding. Linked views let you ask *follow-up questions instantly*. "During the months when Petroleum was high, what happened to Shrimp and Cocoa?" — drag over the Petroleum peaks and the answer appears in the other panels.

Implementation note: True cross-chart brushing requires a reactive framework (Plotly Dash, Altair, D3). Here we implement a **self-contained brushing simulation** using Plotly's `selectedpoints` and `FigureWidget` — the same interaction model, running inside a single notebook cell on Colab.

```
In [15]: # Brushing + Linking: three coordinated Plotly subplots.
# Select a range on the TOP panel (Petroleum time series) using box-select or Lasso;
# the selected months are highlighted in ALL three panels simultaneously.
#
# How it works:
# 1. Each subplot shares the same x-axis (date), so a range selection on any panel
#    maps to the same set of months on every other panel.
# 2. We use relayLayoutData via Plotly's rangeslider to drive a highlighted window.
# 3. On Colab, cross-widget callbacks are not available, so we implement brushing
#    by encoding the year as color + using the shared rangeslider as the brush.

import plotly.graph_objects as go
from plotly.subplots import make_subplots
import pandas as pd

# — Three series to link —————
dates      = df_monthly['date']
petroleum  = df_monthly['Petroleum']
shrimp     = df_monthly['Shrimp']
cocoa      = df_monthly['Cocoa']

# Encode year as color so the brush highlights structure that crosses all panels
year_color = df_monthly['date'].dt.year.map({2023:'#5e4fa2', 2024:'#3288bd', 2025:'#d53e4f'})

fig = make_subplots(
    rows=3, cols=1,
    shared_xaxes=True,          # ← this is the key: all panels share the same x-axis
    subplot_titles=[
        'Petroleum (USD M) — drag the rangeslider below to brush a time window',
        'Shrimp (USD M) — linked: same months highlighted',
        'Cocoa (USD M) — linked: same months highlighted'
    ],
    vertical_spacing=0.08,
    row_heights=[0.38, 0.31, 0.31]
)

# — Add traces: Line + colored scatter dots (color = year) —————
for row, (series, name, lcolor) in enumerate([
    (petroleum, 'Petroleum', '#1d3557'),
    (shrimp, 'Shrimp', '#e07a5f'),
    (cocoa, 'Cocoa', '#7b2d8b')
], start=1):
    # Faint background line
    fig.add_trace(go.Scatter(
        x=dates, y=series, mode='lines',
        line=dict(color=lcolor, width=1.5),
        showlegend=False, hoverinfo='skip'
    ), row=row, col=1)
    # Colored dots (color encodes year — this is what gets 'brushed' visually)
    fig.add_trace(go.Scatter(
        x=dates, y=series,
        mode='markers',
        marker=dict(color=year_color, size=8, line=dict(width=1, color='white')),
        name=name,
        customdata=df_monthly['date'].dt.strftime('%b %Y'),
        hovertemplate='<b>' + name + '</b><br>{customdata}<br>{y:,.0f} USD M<extra></extra>'
    ), row=row, col=1)

# — Legend for year colors —————
for yr, col in [(2023, '#5e4fa2'), (2024, '#3288bd'), (2025, '#d53e4f')]:
    fig.add_trace(go.Scatter(
        x=[None], y=[None], mode='markers',
        marker=dict(color=col, size=10),
        name=yr, showlegend=True
    ))

# — Shared rangeslider on bottom panel acts as the BRUSH —————
fig.update_xaxes(
    rangeslider=dict(visible=True, thickness=0.06),
    rangeselector=dict(buttons=[
```

```

dict(count=6, label='6M', step='month', stepmode='backward'),
dict(count=12, label='1Y', step='month', stepmode='backward'),
dict(step='all', label='All')
]),
row=3, col=1
)

fig.update_layout(
    title=dict(
        text='Brushing + Linking – Petroleum, Shrimp, and Cocoa<br>'
        '<sup>Drag the rangeslider (bottom) to brush a time window – all three panels respond simultaneously</sup>'
        font=dict(size=15)
    ),
    height=680,
    hovermode='x unified',
    legend=dict(title='Year', orientation='h', x=0.01, y=1.03),
    plot_bgcolor='#f8f9fa',
    paper_bgcolor='white'
)

fig.update_yaxes(showgrid=True, gridcolor='#e0e0e0')
fig.show()

print("\nHow to brush:")
print(" 1. Drag the grey rangeslider at the bottom to select a time window.")
print(" 2. All three panels zoom to the same months simultaneously – that is brushing + linking.")
print(" 3. Try selecting just the 2025 months and compare Petroleum vs Cocoa behavior.")
print(" 4. Use the 6M / 1Y buttons for quick brushes.")

```

How to brush:

1. Drag the grey rangeslider at the bottom to select a time window.
2. All three panels zoom to the same months simultaneously – that is brushing + linking.
3. Try selecting just the 2025 months and compare Petroleum vs Cocoa behavior.
4. Use the 6M / 1Y buttons for quick brushes.

Insights

What the brush reveals

Brush over **Sep–Dec 2023** (the petroleum spike period): Shrimp stays flat while Petroleum surges — the two products move independently during that window.

Brush over **2024–2025**: Cocoa dots (purple/red) climb while Petroleum dots flatten — the diversification shift becomes visible only when you isolate the recent window.

This is the core value of brushing: insights that are invisible in the full-range view emerge immediately when you restrict the temporal window.

Part 5 — Storytelling with Data

So far we have *explored* the data. Now we learn **explanatory** visualization: turning a chart into a story that moves an audience to a decision. We follow Cole Nussbaumer Knaflic's *Storytelling with Data* (2015), six lessons applied step by step.

Exploratory vs. explanatory

- **Exploratory** = for *you*, sifting data for pearls (Parts 1–4).
- **Explanatory** = for *your audience*, showing only the pearl that matters.

The Big Idea

- **Audience**: Ecuador's Ministry of Production and Foreign Trade.
- **Message**: Cocoa has quadrupled in value since 2020 — and Ecuador's processed cocoa exports (an industrialized product) are growing even faster, suggesting a path from raw commodity to premium manufactured export.
- **Call to action**: Invest in processing infrastructure and fine-cacao certification to capture more value per kilogram exported.

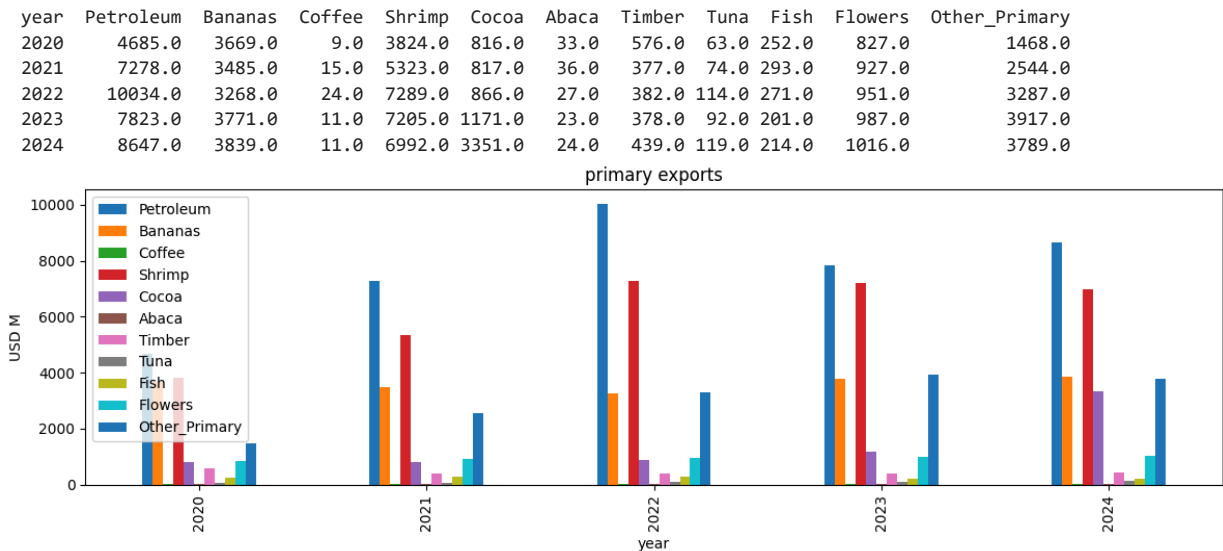
Lesson 1 — Understand the Context

The worst starting point is a data dump with no point of view.

```
In [16]: # Print the raw annual table – the "data dump" starting point with no design intent
print(df_annual[['year'] + primary_products].round(0).to_string(index=False))

# Plot an unformatted bar chart – no color logic, no title meaning, no message
df_annual.set_index('year')[primary_products].plot(kind='bar', figsize=(12, 4))
plt.title('primary exports'); plt.ylabel('USD M'); plt.tight_layout(); plt.show()

# State the context that will guide all design decisions in the next five lessons
print("\nContext: Audience=Ministry | Message=Cocoa quadrupled | CTA=Invest in processing")
```



Context: Audience=Ministry | Message=Cocoa quadrupled | CTA=Invest in processing

Lesson 2 — Choose an Appropriate Display

Line chart beats bar chart for tracking change over time.

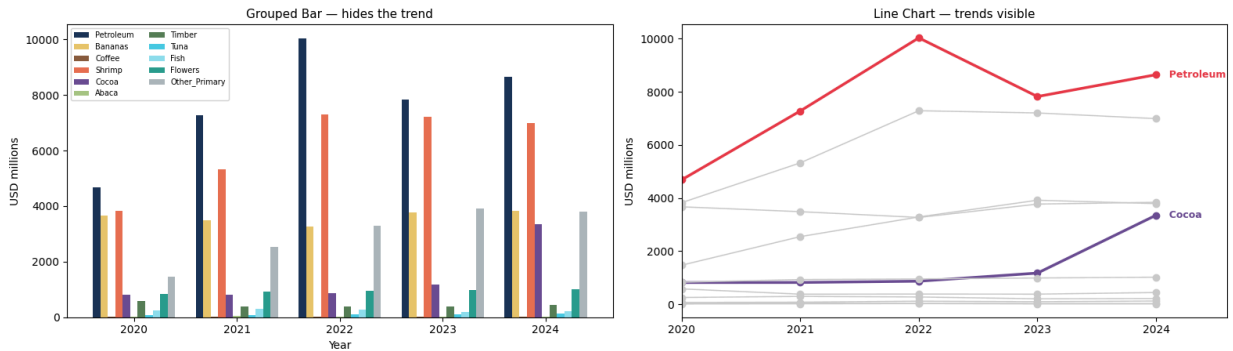
```
In [17]: # Define the two color roles used throughout the storytelling section:
# GRAY = context series (background, not the message)
# HL = highlighted series that carry the story
GRAY = '#cccccc'
HL = {'Cocoa': '#6a4c93', 'Petroleum': '#e63946'}

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))

# — LEFT: wrong choice – grouped bar chart —
# Bar charts show within-year comparisons well but hide the year-over-year trend
df_annual.set_index('year')[primary_products].plot(
    kind='bar', ax=ax1, color=[primary_colors[c] for c in primary_products], width=0.8)
ax1.set_title('Grouped Bar – hides the trend', fontsize=11)
ax1.set_xlabel('Year'); ax1.set_ylabel('USD millions')
ax1.legend(fontsize=7, ncol=2); ax1.tick_params(axis='x', rotation=0)

# — RIGHT: right choice – line chart —
# Line charts make direction and rate of change immediately readable
for cat in primary_products:
    col = HL.get(cat, GRAY); lw = 2.5 if cat in HL else 1.1
    ax2.plot(df_annual['year'], df_annual[cat], marker='o', color=col, linewidth=lw)
    if cat in HL: # only label the highlighted series to reduce clutter
        ax2.text(df_annual['year'].iloc[-1]+0.05, df_annual[cat].iloc[-1],
            '+cat, color=col, fontsize=9, va='center', fontweight='bold')
ax2.set_title('Line Chart – trends visible', fontsize=11)
ax2.set_xticks(df_annual['year']); ax2.set_ylabel('USD millions')
ax2.set_xlim(df_annual['year'].min(), df_annual['year'].max()+0.6) # extra space for labels
plt.suptitle('Lesson 2 – Match the display to the question', fontsize=12, fontweight='bold')
plt.tight_layout(); plt.show()
```

Lesson 2 — Match the display to the question



Lesson 3 — Eliminate Clutter

Direct labels, no legend, no spines.

```
In [18]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))

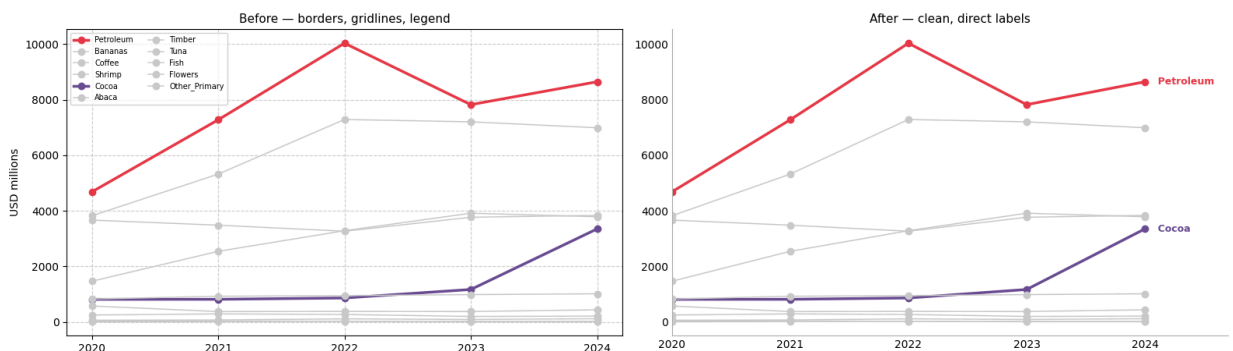
# — LEFT: before — cluttered version —————
for cat in primary_products:
    col = HL.get(cat, GRAY); lw = 2.5 if cat in HL else 1.1
    ax1.plot(df_annual['year'], df_annual[cat], marker='o', color=col, linewidth=lw, label=cat)
ax1.set_title('Before — borders, gridlines, legend', fontsize=11)
ax1.grid(True, linestyle='--', alpha=0.6) # heavy grid adds visual noise
ax1.legend(fontsize=7, ncol=2)           # legend forces eye to jump back and forth
ax1.set_xticks(df_annual['year']); ax1.set_ylabel('USD millions')

# — RIGHT: after — decluttered version —————
for cat in primary_products:
    col = HL.get(cat, GRAY); lw = 2.5 if cat in HL else 1.1
    ax2.plot(df_annual['year'], df_annual[cat], marker='o', color=col, linewidth=lw)
    if cat in HL:
        # Direct label at the end of each highlighted line — replaces the Legend
        ax2.text(df_annual['year'].iloc[-1]+0.05, df_annual[cat].iloc[-1],
                  '+cat', color=col, fontsize=9, va='center', fontweight='bold')

# Remove top and right spines — they frame the data but add no information
for spine in ['top', 'right']:
    ax2.spines[spine].set_visible(False)

# Soften the remaining spines so they recede visually
ax2.spines['left'].set_color('#aaa'); ax2.spines['bottom'].set_color('#aaa')
ax2.tick_params(length=0) # remove tick marks — the grid already shows scale
ax2.set_title('After — clean, direct labels', fontsize=11)
ax2.set_xticks(df_annual['year'])
ax2.set_xlim(df_annual['year'].min(), df_annual['year'].max()+0.7)
plt.suptitle('Lesson 3 — Eliminate clutter', fontsize=12, fontweight='bold')
plt.tight_layout(); plt.show()
```

Lesson 3 — Eliminate clutter



Lesson 4 — Draw Attention Where You Want It

All context series → gray. Message series → color.

```
In [19]: fig, ax = plt.subplots(figsize=(12, 6))

# Color rule: highlighted products get their full color; all others get gray.
# Gray = context (background information); color = message (what the story is about).
```

```

for cat in primary_products:
    col = HL.get(cat, GRAY); lw = 3.0 if cat in HL else 1.2
    ax.plot(df_annual['year'], df_annual[cat], marker='o', color=col, linewidth=lw, zorder=3)
    if cat in HL:
        ax.text(df_annual['year'].iloc[-1]+0.05, df_annual[cat].iloc[-1],
            '+cat, color=col, fontsize=10, va='center', fontweight='bold')

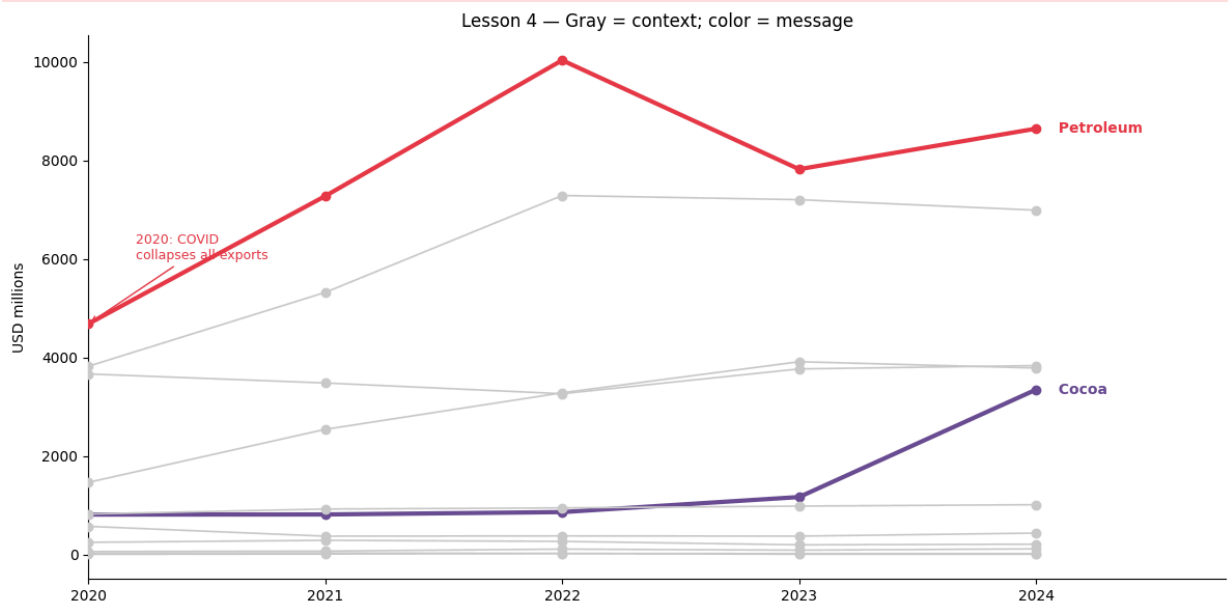
# Annotation: point the reader's attention to the key event (COVID 2020 shock)
# xy = the exact data point the arrow tip lands on
# xytext = where the annotation text is placed
ax.annotate('2020: COVID\ncollapses all exports',
            xy=(2020, float(df_annual[df_annual['year']==2020]['Petroleum'])),
            xytext=(2020.2, 6000), fontsize=9, color='#e63946',
            arrowprops=dict(arrowstyle='->', color='#e63946'))

# Remove distracting spines
for spine in ['top', 'right']:
    ax.spines[spine].set_visible(False)
ax.set_title('Lesson 4 – Gray = context; color = message', fontsize=12)
ax.set_xticks(df_annual['year'])
ax.set_xlim(df_annual['year'].min(), df_annual['year'].max()+0.8)
ax.set_ylabel('USD millions'); plt.tight_layout(); plt.show()

```

/tmp/ipykernel_2299/738479274.py:16: FutureWarning:

Calling float on a single element Series is deprecated and will raise a TypeError in the future. Use float(series.iloc[0]) instead



Lesson 5 — Think Like a Designer

Sentence-style title, single annotation, minimal chrome.

```

In [20]: fig, ax = plt.subplots(figsize=(12, 6.5))

# Same color logic as Lesson 4 – highlighted vs. gray context lines
for cat in primary_products:
    col = HL.get(cat, GRAY); lw = 3.0 if cat in HL else 1.2
    ax.plot(df_annual['year'], df_annual[cat], marker='o', color=col, linewidth=lw, zorder=3)
    if cat in HL:
        ax.text(df_annual['year'].iloc[-1]+0.05, df_annual[cat].iloc[-1],
            '+cat, color=col, fontsize=11, va='center', fontweight='bold')

# Sentence-style title – states the conclusion, not just the topic.
# loc='left' aligns it to the left so it reads like a newspaper headline.
ax.set_title("Cocoa Is Ecuador's Fastest-Growing Primary Export\n"
            "– Petroleum Revenue Has Been Falling Since 2022",
            fontsize=13, fontweight='bold', loc='left', color='#1d3557')

# Remove three spines – only keep the bottom axis line
for spine in ['top', 'right', 'left']:
    ax.spines[spine].set_visible(False)
ax.spines['bottom'].set_color('#999'); ax.tick_params(length=0)

```

```
# Very faint horizontal grid – helps read values without dominating the chart
ax.grid(axis='y', linestyle='-', alpha=0.12)
ax.set_xticks(df_annual['year'])
ax.set_xlim(df_annual['year'].min(), df_annual['year'].max()+1.0)

# Inline units label instead of a y-axis title – less formal, cleaner
ax.text(df_annual['year'].min(), df_annual['Petroleum'].max()*1.03,
        'Annual exports, USD millions (primary products)', fontsize=9, color='#666')

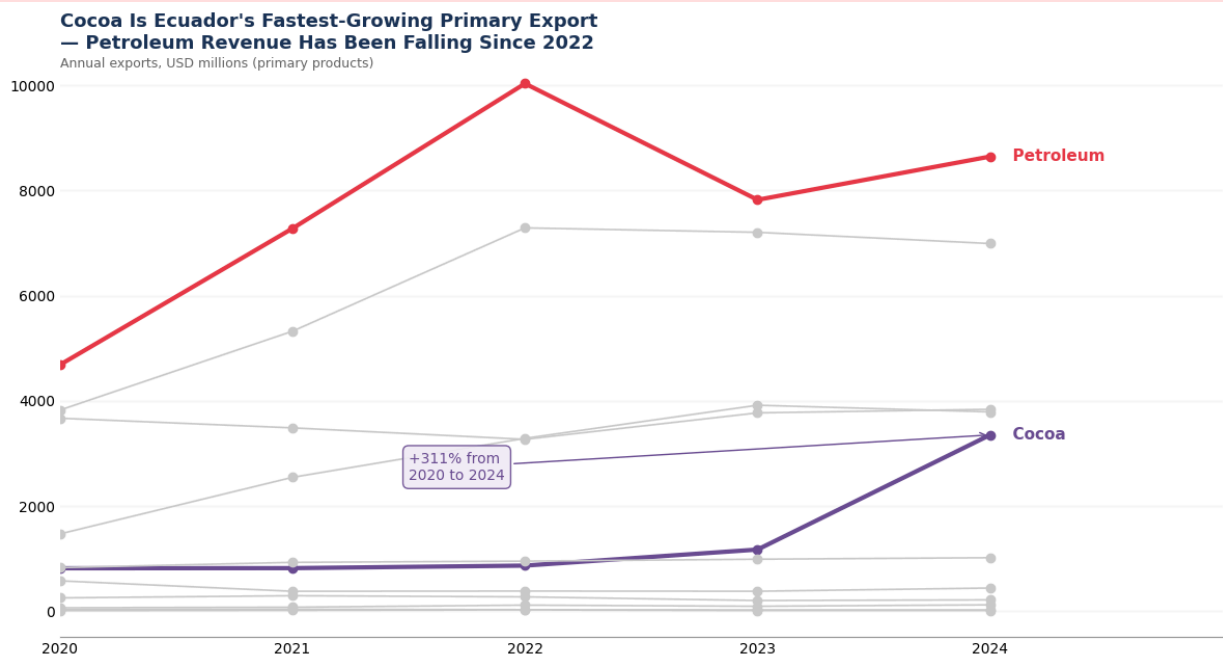
# Annotation box: quantifies the key insight so the reader doesn't have to calculate
c20 = float(df_annual[df_annual['year']==2020]['Cocoa'])
c24 = float(df_annual[df_annual['year']==2024]['Cocoa'])
ax.annotate(f'+{(c24/c20-1)*100:.0f}% from\n2020 to 2024',
            xy=(2024, c24), xytext=(2021.5, 2500), fontsize=10, color='#6a4c93',
            bbox=dict(boxstyle='round,pad=0.4', fc='#f3eef7', ec='#6a4c93', alpha=0.9),
            arrowprops=dict(arrowstyle='->', color='#6a4c93'))
plt.tight_layout(); plt.show()
```

/tmp/ipykernel_2299/2850079894.py:32: FutureWarning:

Calling float on a single element Series is deprecated and will raise a TypeError in the future. Use float(series.iloc[0]) instead

/tmp/ipykernel_2299/2850079894.py:33: FutureWarning:

Calling float on a single element Series is deprecated and will raise a TypeError in the future. Use float(series.iloc[0]) instead



Lesson 6 — Tell a Story

Three sequential panels: setup → complication → resolution.

```
In [21]: # Reusable function that draws one "story slide" panel for a given year subset.
# years_sel: List of years to show (controls how much history is revealed)
# hl: dict of {product: color} for highlighted series; everything else → GRAY
# note: optional annotation text placed on the chart
def story_panel(years_sel, hl, title, note=None, note_xy=None, note_txy=None):
    sub = df_annual[df_annual['year'].isin(years_sel)] # filter to selected years
    fig, ax = plt.subplots(figsize=(11, 5))
    for cat in primary_products:
        col = hl.get(cat, GRAY); lw = 3.0 if cat in hl else 1.2
        ax.plot(sub['year'], sub[cat], marker='o', color=col, linewidth=lw, zorder=3)
        if cat in hl:
            ax.text(sub['year'].iloc[-1]+0.05, sub[cat].iloc[-1],
                    '+cat, color=col, fontsize=10, va='center', fontweight='bold')
    ax.set_title(title, fontsize=13, fontweight='bold', loc='left', color='#1d3557')
    for spine in ['top', 'right', 'left']:
        ax.spines[spine].set_visible(False)
    ax.spines['bottom'].set_color('#999'); ax.tick_params(length=0)
    ax.grid(axis='y', linestyle='-', alpha=0.12)
    ax.set_xticks(years_sel)
```

```

ax.set_xlim(min(years_sel)-0.1, max(years_sel)+0.9)
# Fixed y-axis across all panels so the scale is consistent across "slides"
ax.set_ylim(0, df_annual[primary_products].max().max()*1.12)
if note: # optional annotation pointing to a specific data point
    ax.annotate(note, xy=note_xy, xytext=note_txy, fontsize=10, color='#6a4c93',
                arrowprops=dict(arrowstyle='->', color='#6a4c93'))
plt.tight_layout(); plt.show()

# Panel A – SETUP: show only 2020-2021 to establish petroleum's dominance
story_panel([2020, 2021], {'Petroleum': '#e63946'},
            "1. Petroleum Dominated Ecuador's Primary Exports – Then COVID Hit")

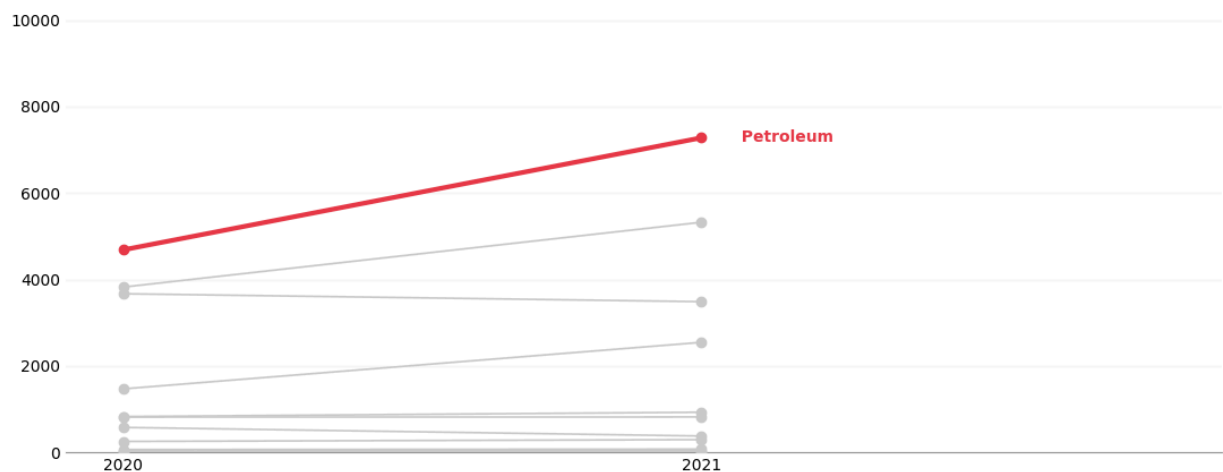
# Panel B – COMPLICATION: extend to 2023 to reveal cocoa's quiet rise
story_panel([2020, 2021, 2022, 2023], {'Petroleum': '#e63946', 'Cocoa': '#6a4c93'},
            "2. Recovery Was Uneven: Cocoa Began a Quiet but Consistent Climb",
            note='Cocoa doubles\nfrom 2020 to 2023',
            note_xy=(2023, float(df_annual[df_annual['year']==2023]['Cocoa'])),
            note_txy=(2021.2, 2500))

# Panel C – RESOLUTION: show the full picture through 2024
story_panel([2020, 2021, 2022, 2023, 2024], {'Petroleum': '#e63946', 'Cocoa': '#6a4c93'},
            "3. By 2024, Cocoa Has Quadrupled – Ecuador's Clearest Diversification Signal")

# Policy recommendation – the explicit call to action that the story argues for
print("=" * 70)
print("POLICY RECOMMENDATION")
print("=" * 70)
print("Ecuador should invest in fine-cacao processing, origin certification,")
print("and direct-trade relationships – capturing more value per kilogram")
print("rather than exporting raw beans at commodity prices.")

```

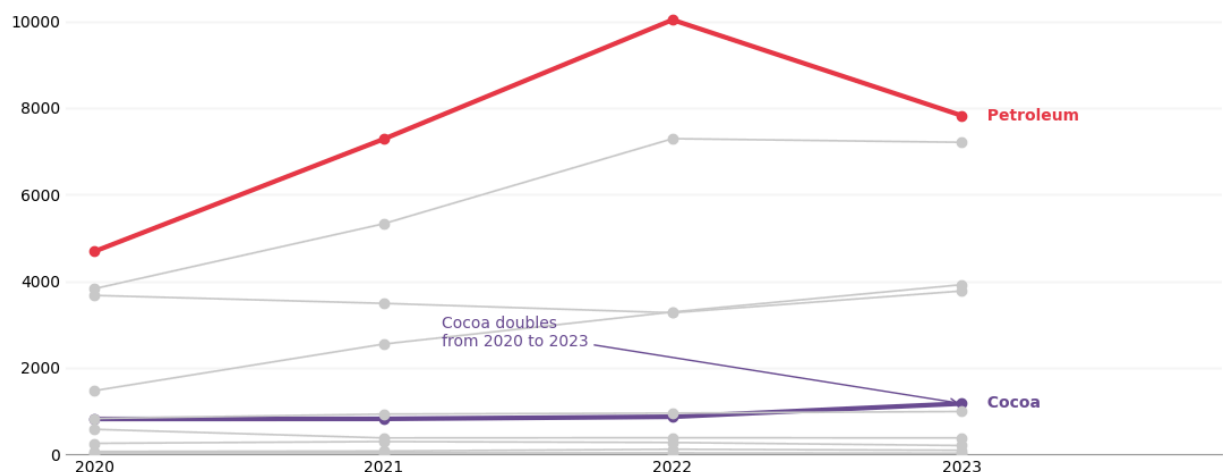
1. Petroleum Dominated Ecuador's Primary Exports – Then COVID Hit



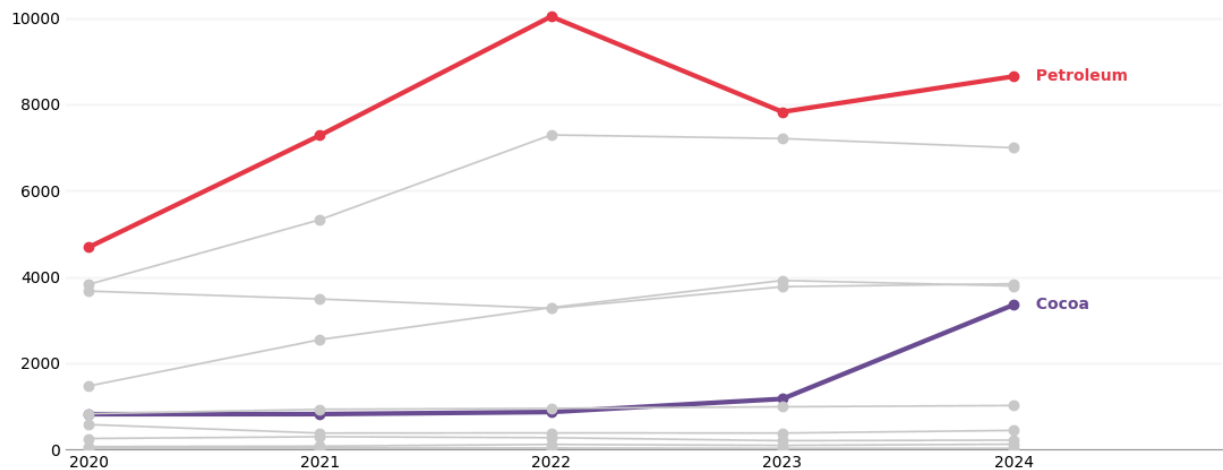
/tmp/ipykernel_2299/1764246069.py:36: FutureWarning:

Calling float on a single element Series is deprecated and will raise a TypeError in the future. Use float(series.iloc[0]) instead

2. Recovery Was Uneven: Cocoa Began a Quiet but Consistent Climb



3. By 2024, Cocoa Has Quadrupled — Ecuador's Clearest Diversification Signal



POLICY RECOMMENDATION

Ecuador should invest in fine-cacao processing, origin certification, and direct-trade relationships – capturing more value per kilogram rather than exporting raw beans at commodity prices.

Part 6 — Activities

Parts 1–5 explored Ecuador's **primary exports** (PRIMARIOS) — the raw commodities that form the backbone of the country's trade. In these activities you apply the same visualization techniques to Ecuador's **industrialized exports** (INDUSTRIALIZADOS) — the ten categories of processed and manufactured goods. This lets you practice each technique on a new product group and draw comparisons between the two halves of Ecuador's export economy.

Complete each activity independently using `df_monthly` (or `df_annual` where indicated). **Each activity is worth the same number of points.**

Where a task asks for an interpretation, write your answer in a markdown cell after your code.

Activity 1: Time Series Decomposition — Industrialized Products

In Part 1 we decomposed **Petroleum** (a primary product). Now apply the same technique to **Processed Cocoa** (`Processed_Cocoa`), an industrialized product that directly derives from Ecuador's fastest-growing primary export.

Using `df_monthly`:

1. Plot the raw monthly series of `Processed_Cocoa` as a line chart.
2. Compute and overlay a **3-month centered rolling mean** to approximate the trend.
3. Compute the **deviation from trend** (raw – rolling mean) and plot it in a second panel.
4. In a markdown cell: does Processed Cocoa show a seasonal pattern? How does its pattern compare to raw Cocoa (from Part 1's decomposition example)? What does this tell you about the relationship between the primary and industrialized stages of the cocoa value chain?

```
In [22]: pc = df_monthly.set_index('date')['Processed_Cocoa']

# TREND: 3-month centered rolling mean (shorter window than Petroleum's 6-month, appropriate since Processed_Cocoa
pc_trend = pc.rolling(window=3, center=True).mean()

# DEVIATION FROM TREND: raw - rolling mean (seasonality + noise combined, since the activity only asks for a 2-panel
pc_deviation = pc - pc_trend

fig, axes = plt.subplots(2, 1, figsize=(13, 6), sharex=True)
fig.suptitle("Activity 1 – Decomposition of Ecuador's Processed Cocoa Exports\n"
            "(Jan 2023 – Oct 2025, real BCE data, 3-month rolling trend)",
            fontsize=13, fontweight='bold')

# Panel 1: raw observed series + trend overlay
axes[0].plot(pc.index, pc.values, color=ind_colors['Processed_Cocoa'],
```

```

        linewidth=1.6, marker='o', markersize=3, label='Observed')
axes[0].plot(pc_trend.index, pc_trend.values, color='#e63946',
            linewidth=2, linestyle='--', label='3-mo rolling trend')
axes[0].set_ylabel('USD M')
axes[0].legend(fontsize=9)
axes[0].grid(True, linestyle='--', alpha=0.4)

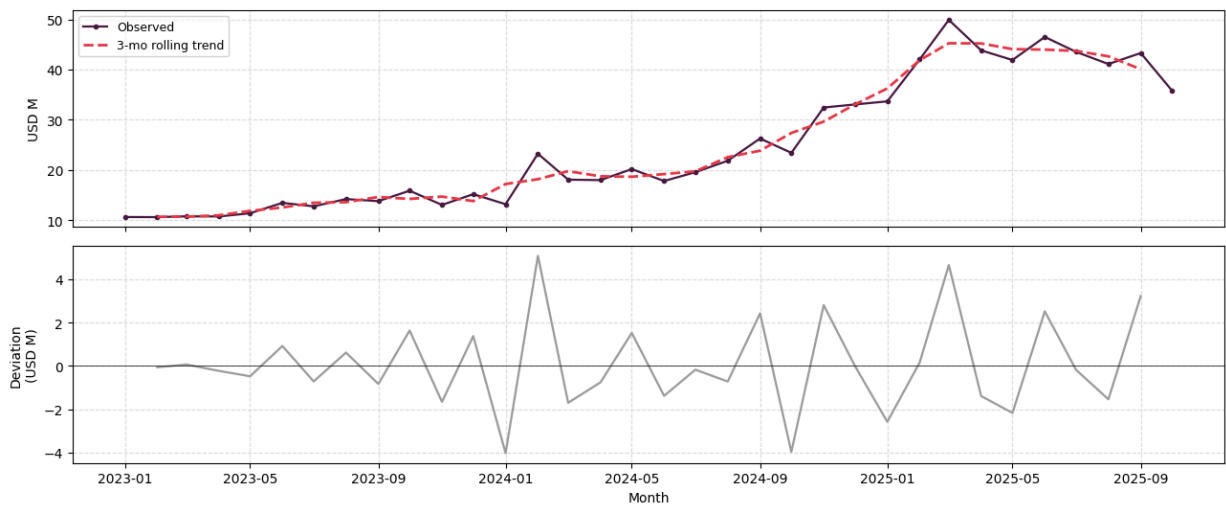
# Panel 2: deviation from trend
axes[1].plot(pc_deviation.index, pc_deviation.values, color='#9e9e9e', linewidth=1.6)
axes[1].axhline(0, color='black', linewidth=0.5)
axes[1].set_ylabel('Deviation\n(USD M)')
axes[1].set_xlabel('Month')
axes[1].grid(True, linestyle='--', alpha=0.4)

plt.tight_layout()
plt.show()

# Quick numeric summary to support the interpretation
print(f"Processed Cocoa – mean: {pc.mean():.1f} USD M | std: {pc.std():.1f} USD M | "
      f"coefficient of variation: {pc.std()/pc.mean()*100:.1f}%")

```

**Activity 1 — Decomposition of Ecuador's Processed Cocoa Exports
(Jan 2023 – Oct 2025, real BCE data, 3-month rolling trend)**



Processed Cocoa – mean: 24.7 USD M | std: 12.9 USD M | coefficient of variation: 52.2%

Processed Cocoa does not show a clear and consistently repeating seasonal pattern. Although the seasonal component fluctuates between approximately –4 and +5 million USD, the peaks and declines do not occur in the same months each year. For example, October 2023 shows a pronounced decline, whereas October 2024 records an increase, and additional peaks appear in March and September 2025. Therefore, the series is characterized more strongly by its upward trend than by seasonality. Monthly export values increased from approximately USD 11 million in January 2023 to nearly USD 50 million at the beginning of 2025.

This pattern is similar to that of raw Cocoa analyzed in Part 1. Both raw and Processed Cocoa exhibit strong growth and considerable short-term fluctuations, but neither presents a stable monthly pattern that repeats clearly across the years. Raw Cocoa also showed one of the largest increases among primary products, with substantial variation across the period. Processed Cocoa appears to follow a broadly similar trajectory, suggesting that the growth of cocoa processing has occurred alongside the expansion of raw cocoa exports.

The similarity between the two series indicates that the primary and industrialized stages of Ecuador's cocoa value chain are closely connected. As the production and export value of raw Cocoa increase, Processed Cocoa exports also tend to expand, suggesting that the industrial stage is benefiting from the growth of the primary sector rather than evolving independently from it. However, this visual similarity demonstrates association, not causation, and should ideally be confirmed through a direct comparison of growth rates, market shares, and correlations between both series.

Overall, the results suggest that Ecuador is beginning to capture more value through cocoa processing, but the industrialized stage remains considerably smaller than the primary stage. This means that much of the potential value added is still not retained domestically and supports the recommendation to strengthen processing infrastructure, innovation, certification, and access to higher-value international markets.



Activity 2: Visualization Taxonomies — All 10 Industrialized Products

In Part 2 we applied shared-space and split-space taxonomies to primary products. Now apply both to **all 10 industrialized products**.

Using `df_monthly`:

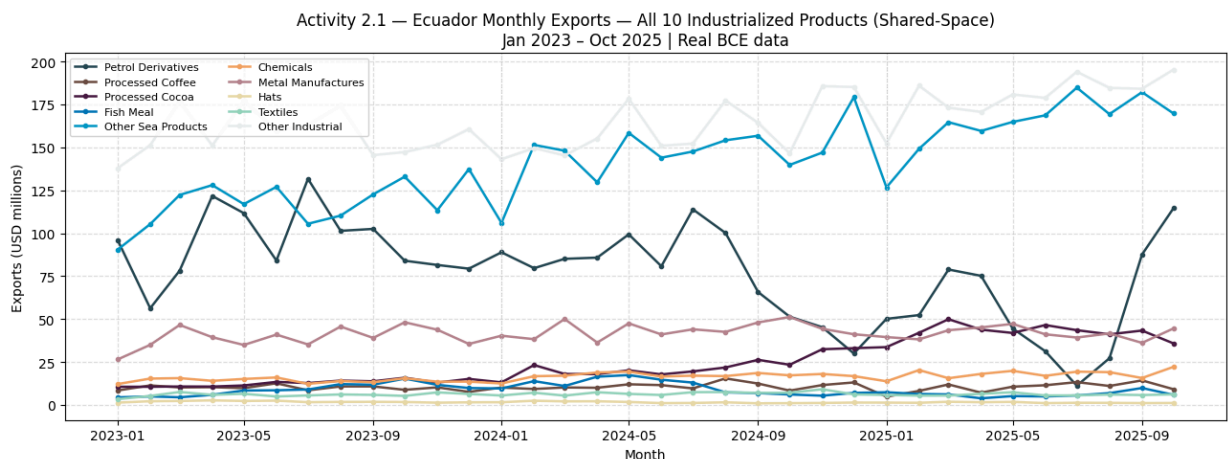
1. Build a **shared-space line chart** with all 10 industrialized products on one axis. Use `ind_colors` for the color palette.
2. Build a **split-space small-multiples** grid (2x5) with one panel per industrialized product, `sharey=True`.
3. In a markdown cell: which taxonomy works better for industrialized products? Is the scale problem (dominant product compressing the others) worse or better than it was for primary products, and why?

```
In [23]: # Activity 2.1 – Shared-space: ALL 10 industrialized products on one axis
fig, ax = plt.subplots(figsize=(13, 5))
for cat in industrialized_products:
    ax.plot(df_monthly['date'], df_monthly[cat],
            label=cat.replace('_', ' '), color=ind_colors[cat],
            linewidth=1.8, marker='o', markersize=3)

for yr in [2024, 2025]:
    ax.axvline(pd.Timestamp(f'{yr}-01-01'), color='#ccc', linewidth=0.8, linestyle=':')

ax.set_title("Activity 2.1 – Ecuador Monthly Exports – All 10 Industrialized Products (Shared-Space)\n"
            "Jan 2023 – Oct 2025 | Real BCE data", fontsize=12)
ax.set_xlabel('Month'); ax.set_ylabel('Exports (USD millions)')
ax.legend(loc='upper left', fontsize=8, ncol=2)
ax.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout(); plt.show()

print("Compare: does one product dominate and compress the others, the way")
print("Petroleum did for primary products in Part 2.1?")
```



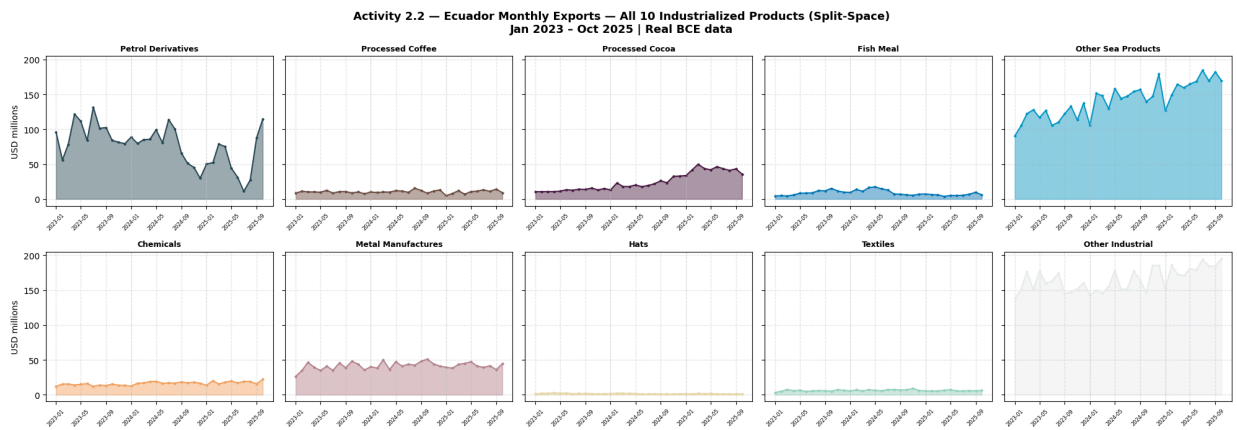
Compare: does one product dominate and compress the others, the way
Petroleum did for primary products in Part 2.1?

```
In [24]: # Activity 2.2 – Split-space: ALL 10 industrialized products, 2x5 grid
fig, axes = plt.subplots(2, 5, figsize=(20, 7), sharey=True)
fig.suptitle("Activity 2.2 – Ecuador Monthly Exports – All 10 Industrialized Products (Split-Space)\n"
            "Jan 2023 – Oct 2025 | Real BCE data", fontsize=13, fontweight='bold')

for ax, cat in zip(axes.flat, industrialized_products):
    ax.fill_between(df_monthly['date'], df_monthly[cat],
                   alpha=0.45, color=ind_colors[cat])
    ax.plot(df_monthly['date'], df_monthly[cat],
            color=ind_colors[cat], linewidth=1.4, marker='o', markersize=2)
    for yr in [2024, 2025]:
        ax.axvline(pd.Timestamp(f'{yr}-01-01'), color='#ccc', linewidth=0.8, linestyle=':')
    ax.set_title(cat.replace('_', ' '), fontsize=9, fontweight='bold')
    ax.tick_params(axis='x', rotation=45, labels=6)
    ax.grid(True, linestyle='--', alpha=0.3)

for row in range(2):
    axes[row, 0].set_ylabel('USD millions')

plt.tight_layout()
plt.show()
```



Activity 3: Heatmap — All 10 Industrialized Products

In Part 3.1 we built a heatmap comparing Total Primary vs. Total Industrialized. Now drill into the industrialized group itself.

Using `df_monthly`:

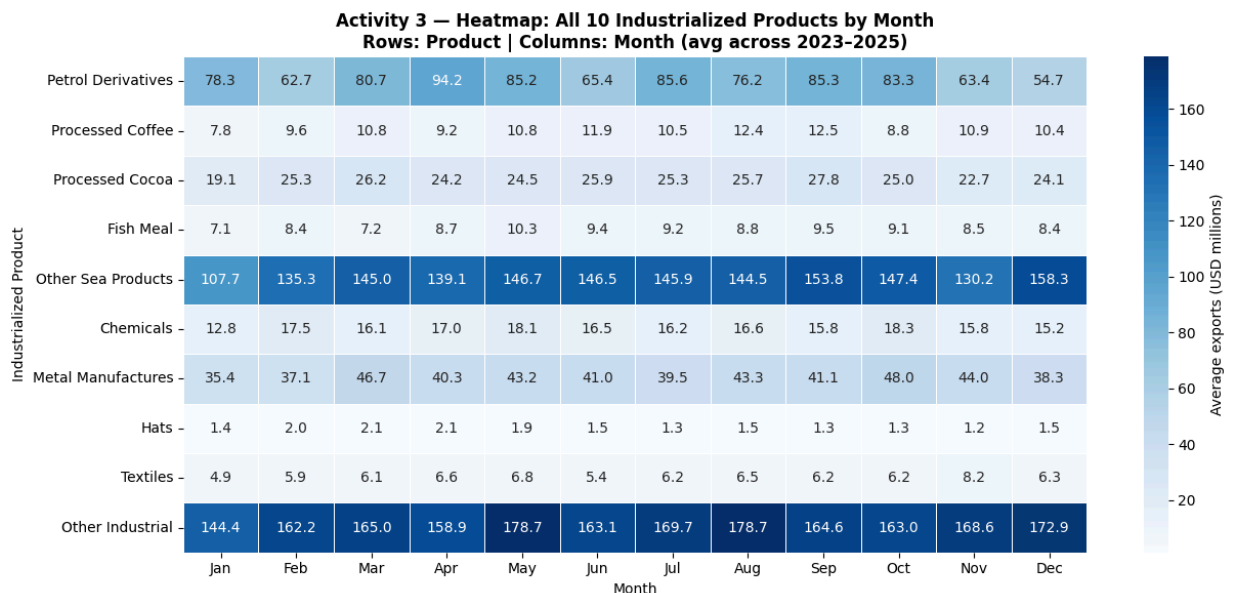
1. Build a **heatmap** with rows = the 10 industrialized products, columns = month (Jan–Dec), color = average export value in USD millions. Use the `Blues` colormap.
2. Annotate cells with values (`annot=True` , `fmt='.1f'`).
3. In a markdown cell: which industrialized products have the strongest seasonal patterns? Compare with the primary product heatmap from Part 3.1 — does the industrialized group show more or less seasonality overall?

```
In [25]: month_labels = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                        'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

# Pivot: mean value per product per calendar month, transposed so products are rows and months are columns (as the primary product heatmap)
pivot_ind = df_monthly.groupby('month')[industrialized_products].mean().T
pivot_ind.columns = month_labels
pivot_ind.index = [c.replace('_', ' ') for c in pivot_ind.index]

fig, ax = plt.subplots(figsize=(13, 6))
sns.heatmap(pivot_ind, cmap='Blues', annot=True, fmt='.1f',
            linewidths=0.4, linecolor='white',
            cbar_kws={'label': 'Average exports (USD millions)'}, ax=ax)
ax.set_title("Activity 3 – Heatmap: All 10 Industrialized Products by Month\n"
            "Rows: Product | Columns: Month (avg across 2023-2025)", fontsize=12, fontweight='bold')
ax.set_xlabel('Month'); ax.set_ylabel('Industrialized Product')
plt.tight_layout()
plt.show()

# Quantify seasonal strength per product: range (max - min month) as a % of that product's own average – a product's own average
seasonal_strength = ((pivot_ind.max(axis=1) - pivot_ind.min(axis=1)) / pivot_ind.mean(axis=1) * 100).sort_values(ascending=False)
print("Seasonal strength (month-to-month range as % of average) – highest first:")
print(seasonal_strength.round(1))
```



Seasonal strength (month-to-month range as % of average) – highest first:

Hats	56.0
Textiles	53.1
Petrol Derivatives	51.9
Processed Coffee	45.2
Fish Meal	36.5
Other Sea Products	35.7
Processed Cocoa	35.0
Chemicals	33.7
Metal Manufactures	30.2
Other Industrial	20.7

dtype: float64

The industrialized products with the strongest seasonal patterns are Hats, Textiles, and Petroleum Derivatives, with seasonal variation of approximately 56.5%, 52.6%, and 51.8%, respectively. In each case, the highest monthly value is nearly twice the lowest one in relative terms. Petroleum Derivatives show a particularly clear pattern, with a major peak in April and another in September, followed by a pronounced decline in November and December. This resembles the seasonal behavior of crude Petroleum observed earlier, especially the September peak and the year-end decline, which is reasonable because both products belong to different stages of the same value chain. At the opposite extreme, Other Industrialized Products show very weak seasonality, with values remaining relatively stable throughout the year and no clear recurring calendar pattern.

Compared with the primary-product heatmap from Part 3.1, the industrialized group appears less seasonal when viewed only at the aggregate level. However, the product-level analysis reveals a more complex picture. Seven of the ten individual industrialized products display moderate to strong seasonal variation, in some cases comparable to or greater than that of several primary products. The key difference is that their peaks occur in different months. For example, Petroleum Derivatives peak in April and September, Other Sea Products in September and December, and Textiles in May and November.

Because these seasonal peaks are not synchronized, they tend to offset one another when all products are combined into the Total Industrialized series. As a result, the aggregate heatmap appears relatively flat even though many individual components have clear seasonal patterns. This is a case of aggregation cancellation: the absence of visible seasonality in the total does not mean that the individual products lack seasonality, but rather that their cycles occur at different times and partially cancel each other out.

Therefore, the industrialized group shows less seasonality overall at the aggregate level, but substantial seasonality at the individual-product level. This also demonstrates why detailed space-filling views are necessary to complement aggregated heatmaps. Just as split-space revealed patterns hidden in the shared-space chart, the product-level heatmap uncovers seasonal behavior that is obscured when all industrialized products are summed together.

Activity 4: Interactive Chart — All 10 Industrialized Products

Build an **interactive Plotly line chart** of all 10 industrialized products from `df_monthly` that:

1. Uses `ind_colors` for colors, hover tooltips (product name, date, value in USD M), and `hovermode='x unified'`.
2. Adds a **range selector** (6M, 1Y, All) and a **range slider**.
3. In a markdown cell: use the 1Y zoom to focus on 2024. Which industrialized product was most volatile month-to-month? Which showed the most consistent upward trend? How does the volatility of industrialized products compare to what you observed for primary products in Part 4.1?

```
In [26]: fig = go.Figure()
for cat in industrialized_products:
    fig.add_trace(go.Scatter(
        x=df_monthly['date'], y=df_monthly[cat],
        mode='lines+markers',
        name=cat.replace('_', ' '),
        line=dict(color=ind_colors[cat], width=1.8),
        marker=dict(size=3),
        hovertemplate='<b>' + cat.replace('_', ' ') + '</b><br>{x:%b %Y}<br>{y:,.1f} USD M<extra></extra>'
    ))

fig.update_layout(
    title="Activity 4 – Ecuador Monthly Industrialized Exports – All 10 Products (Interactive)<br>"
        "<sup>Click legend to filter | Drag to zoom | 6M/1Y buttons | Hover for exact values</sup>",
    xaxis_title='Month', yaxis_title='Exports (USD millions)',
    hovermode='x unified',
    height=540,
    legend=dict(x=1.01, y=1),
    xaxis=dict(
        rangelslider=dict(visible=True),
        rangeselector=dict(buttons=[
            dict(count=6, label='6M', step='month', stepmode='backward'),
            dict(count=1, label='1Y', step='year', stepmode='backward'),
```

```
dict(step='all', label='All'))))
fig.show()
```

```
In [27]: # Support code: quantify volatility and trend consistency for 2024 specifically, since the markdown question asks
m24_ind = df_monthly[df_monthly['year'] == 2024]

print("2024 monthly volatility and trend – Industrialized products")
print(f"{'Product':<22}{'CV (%)':>10}{'Trend slope':>15}{'R² (fit)':>12}")
for cat in industrialized_products:
    series = m24_ind[cat].values
    cv = series.std() / series.mean() * 100
    x = np.arange(len(series))
    slope, intercept = np.polyfit(x, series, 1)
    pred = slope * x + intercept
    ss_res = ((series - pred) ** 2).sum()
    ss_tot = ((series - series.mean()) ** 2).sum()
    r2 = 1 - ss_res / ss_tot if ss_tot > 0 else 0
    print(f"{'cat.replace('_', ' '):<22}{'cv:>9.1f}%{'slope:>15.2f}{'r2:>12.2f}'")
```

Product	CV (%)	Trend slope	R ² (fit)
Petrol Derivatives	30.6%	-4.40	0.41
Processed Coffee	16.8%	0.21	0.16
Processed Cocoa	25.6%	1.35	0.67
Fish Meal	38.0%	-0.76	0.42
Other Sea Products	11.4%	2.91	0.36
Chemicals	9.7%	0.18	0.14
Metal Manufactures	10.2%	0.41	0.10
Hats	31.1%	-0.10	0.49
Textiles	14.7%	0.14	0.23
Other Industrial	9.6%	2.93	0.43

Using the 1Y zoom to focus on 2024, Fish Meal was the most volatile industrialized product on a month-to-month basis, with a coefficient of variation of approximately 38.0%. It was followed by Hats at 31.1% and Petroleum Derivatives at 30.6%. This is not immediately obvious from the interactive chart because Fish Meal operates on a much smaller scale, generally below USD 20 million, while larger categories such as Other Sea Products produce more visually noticeable movements. For this reason, the coefficient of variation is a more appropriate measure than visual inspection alone, since it evaluates volatility relative to each product's own average.

The industrialized product with the most consistent upward trend was Processed Cocoa. It had a positive monthly slope of approximately 1.35 and the highest coefficient of determination, with an (R^2) of 0.67. This indicates that Processed Cocoa did not simply increase during 2024, but followed a comparatively steady and orderly upward trajectory, with less fluctuation around the trend line. This finding is consistent with the earlier decomposition analysis, which showed that the series was dominated by a strong trend rather than by a stable seasonal pattern.

Compared with the primary products analyzed in Part 4.1, industrialized products were generally less volatile in 2024. Seven of the ten industrialized categories had coefficients of variation below 20%, suggesting a relatively stable month-to-month performance. In the primary group, by contrast, Petroleum showed much more abrupt and unpredictable movements, including the sharp shock observed in July 2025. However, volatility within the industrialized group was not uniform. Petroleum Derivatives also displayed a pronounced decline toward the end of 2024, followed by a strong recovery later in the series, suggesting that some of the instability of crude Petroleum is transmitted to the industrialized stage of the same value chain.

Overall, the results suggest that industrialized products tend to be more stable than primary commodities, but this depends on the type of product. Industrialized goods that remain closely linked to volatile commodities, such as Petroleum Derivatives, can inherit much of the instability of the primary stage. In contrast, more differentiated manufactured products, including Textiles, Chemicals, and Metal Manufactures, appear considerably more stable. This supports the argument that diversification should focus not only on industrialization in general, but specifically on manufacturing activities that generate higher value added and are less exposed to commodity-price volatility.

Activity 5: Parallel Coordinates — All 10 Industrialized Products

In Part 3.6 we built parallel coordinates for all 11 primary products. Now apply the same technique to the 10 industrialized products.

Using `df_monthly`:

1. Build a **Plotly parallel coordinates** chart with all 10 industrialized products as axes. Color by year (2023=teal, 2024=yellow, 2025=red). Use `ind_colors` values as the colorscale where possible, or use the year-based colorscale from Part 3.6.

2. In a markdown cell: identify one **positive correlation** and one **negative correlation** between industrialized products. Do the correlated pairs make economic sense? Compare the overall structure of this chart with the primary products' parallel coordinates in Part 3.6 — which group shows more correlated co-movement?

```
In [28]: fig = px.parallel_coordinates(
    df_monthly,
    dimensions=industrialized_products,
    color='year',
    color_continuous_scale=[[0.0, '#2a9d8f'], [0.5, '#e9c46a'], [1.0, '#e63946']],
    labels={c: c.replace('_', ' ') for c in industrialized_products},
    title='Activity 5 – Parallel Coordinates – All 10 Industrialized Products by Month (2023–2025)'
        '<sup>Each line = one month | Teal=2023 | Yellow=2024 | Red=2025</sup>'
)
fig.update_layout(height=550)
fig.show()

# Correlation highlights to support the written interpretation
print("Correlation highlights (10 industrialized products):")
corr_ind = df_monthly[industrialized_products].corr().round(2)
pairs_ind = [(r, c, corr_ind.loc[r, c]) for r in industrialized_products for c in industrialized_products
              if r < c and (corr_ind.loc[r, c] > 0.5 or corr_ind.loc[r, c] < -0.3)]
pairs_ind.sort(key=lambda x: abs(x[2]), reverse=True)
for r, c, v in pairs_ind[:8]:
    direction = 'positive' if v > 0 else 'negative'
    print(f" {r} ↔ {c}: {v:+.2f} ({direction})")

# Also compute overall "co-movement strength" for comparison with primary products: average absolute correlation
avg_abs_corr_ind = corr_ind.where(~np.eye(len(industrialized_products), dtype=bool)).abs().mean().mean()
corr_prim = df_monthly[primary_products].corr()
avg_abs_corr_prim = corr_prim.where(~np.eye(len(primary_products), dtype=bool)).abs().mean().mean()
print(f"\nAverage |correlation| – Industrialized: {avg_abs_corr_ind:.2f} | Primary: {avg_abs_corr_prim:.2f}")
```

Correlation highlights (10 industrialized products):
Other_Sea_Products ↔ Processed_Cocoa: +0.81 (positive)
Chemicals ↔ Other_Sea_Products: +0.72 (positive)
Other_Industrial ↔ Other_Sea_Products: +0.66 (positive)
Other_Industrial ↔ Processed_Cocoa: +0.65 (positive)
Chemicals ↔ Other_Industrial: +0.61 (positive)
Petrol_Derivatives ↔ Processed_Cocoa: -0.60 (negative)
Chemicals ↔ Processed_Cocoa: +0.58 (positive)
Other_Sea_Products ↔ Petrol_Derivatives: -0.48 (negative)

Average |correlation| – Industrialized: 0.30 | Primary: 0.17

A clear positive correlation appears between Other Sea Products and Processed Cocoa, with a coefficient of approximately +0.81, the strongest positive relationship in the industrialized group. However, this pair does not have an obvious direct productive link, since processed seafood and processed cocoa do not share the same inputs, harvesting cycle, or value chain. The relationship is more likely explained by a common upward trend: both categories increased steadily between 2023 and 2025, so they move together over time even though one does not necessarily cause the other to grow. A similar interpretation applies to other positive pairs, such as Chemicals and Other Sea Products.

The strongest negative correlation is between Petroleum Derivatives and Processed Cocoa, at approximately -0.60. This relationship is economically interpretable, but not because the two products directly compete. Rather, they followed opposite trajectories during the period: Processed Cocoa grew consistently, while Petroleum Derivatives declined and experienced a sharp fall toward the end of 2024. Their negative correlation therefore reflects two contrasting export trends occurring at the same time—expansion in the cocoa value chain and contraction in petroleum-based industrial exports.

The correlated pairs make partial economic sense, but they should not automatically be interpreted as causal relationships. The positive correlations are likely driven mainly by shared trends and common macroeconomic conditions rather than direct production links. In contrast, the negative relationship involving Petroleum Derivatives is meaningful as a description of structural divergence: one industrialized sector was expanding while another was losing momentum. Correlation in this chart should therefore be understood as co-movement, not proof that changes in one product directly cause changes in another.

Compared with the parallel-coordinates chart for primary products in Part 3.6, the industrialized group shows stronger overall co-movement. The average absolute correlation is approximately 0.30 for industrialized products, compared with about 0.17 for primary products. This difference is economically plausible because primary commodities such as Petroleum, Bananas, Shrimp, Cocoa, and Coffee are affected by distinct harvest cycles, international prices, climatic conditions, and supply shocks. Industrialized products, by contrast, may respond more similarly to common factors such as manufacturing activity, investment, export demand, logistics, and conditions in shared destination markets. As a result, their lines tend to move together more frequently, even when there is no direct productive connection between the individual products.

Activity 6: Storytelling — Build a Narrative About Industrialized Exports

Apply Knaflic's six lessons (demonstrated in Part 5 using primary products) to build a persuasive chart about **one or two industrialized products**.

1. Write a one-sentence **Big Idea** (audience + message + call to action) in a markdown cell.
2. Choose **one or two industrialized products** that tell an interesting story. Build a single polished chart using `df_annual` (for the 2020–2024 arc) applying all six lessons:
 - Line chart (appropriate display for trends)
 - Decluttered (no border spines, no legend — use direct labels)
 - Focused attention (your chosen product(s) in color, rest in `#cccccc` (gray color))
 - Sentence-style title stating the conclusion
 - One annotation naming the key quantitative insight
3. In a markdown cell: what policy action does your chart argue for? How does the story of your chosen industrialized product complement or contrast with the cocoa narrative from Part 5?

```
In [29]: # Activity 6: Storytelling – Industrialized exports
# Big idea: Ecuador should invest in cocoa processing because Processed_Cocoa is the clearest value-added industry
# Chemicals is shown as a secondary industrial benchmark: steady growth, but a less dramatic narrative than cocoa

GRAY = '#cccccc'
HL = {
    'Processed_Cocoa': '#4a1942',
    'Chemicals': '#f4a261',
}

fig, ax = plt.subplots(figsize=(12, 6.5))

for product in industrialized_products:
    color = HL.get(product, GRAY)
    linewidth = 3.0 if product in HL else 1.2
    ax.plot(
        df_annual['year'],
        df_annual[product],
        marker='o',
        color=color,
        linewidth=linewidth,
        zorder=3,
    )
    if product in HL:
        ax.text(
            df_annual['year'].iloc[-1] + 0.06,
            df_annual[product].iloc[-1],
            ' ' + product.replace('_', ' '),
            color=color,
            fontsize=10,
            va='center',
            fontweight='bold',
        )

# Quantify the key point so the chart tells a concrete story.
pc_2020 = float(df_annual.loc[df_annual['year'] == 2020, 'Processed_Cocoa'].iloc[0])
pc_2024 = float(df_annual.loc[df_annual['year'] == 2024, 'Processed_Cocoa'].iloc[0])
chem_2020 = float(df_annual.loc[df_annual['year'] == 2020, 'Chemicals'].iloc[0])
chem_2024 = float(df_annual.loc[df_annual['year'] == 2024, 'Chemicals'].iloc[0])

ax.annotate(
    f'Processed Cocoa +{(pc_2024 / pc_2020 - 1) * 100:.0f}% from 2020 to 2024',
    xy=(2024, pc_2024),
    xytext=(2021.35, pc_2024 * 1.35),
    fontsize=10,
    color='#4a1942',
    bbox=dict(boxstyle='round,pad=0.4', fc='#f3eef7', ec='#4a1942', alpha=0.9),
    arrowprops=dict(arrowstyle='->', color='#4a1942'),
)

ax.annotate(
    f'Chemicals +{(chem_2024 / chem_2020 - 1) * 100:.0f}% from 2020 to 2024',
    xy=(2024, chem_2024),
    xytext=(2021.45, chem_2024 * 2.8),
    fontsize=9,
    color='#b36b2c',
    bbox=dict(boxstyle='round,pad=0.35', fc='#fff4e8', ec='#b36b2c', alpha=0.9),
)
```



```

        arrowprops=dict(arrowstyle='->', color='#b36b2c'),
    )

    for spine in ['top', 'right', 'left']:
        ax.spines[spine].set_visible(False)
    ax.spines['bottom'].set_color('#999')
    ax.tick_params(length=0)
    ax.grid(axis='y', linestyle='-', alpha=0.12)
    ax.set_xticks(df_annual['year'])
    ax.set_xlim(df_annual['year'].min() - 0.1, df_annual['year'].max() + 1.0)
    ax.set_ylim(0, df_annual[industrialized_products].max().max() * 1.15)

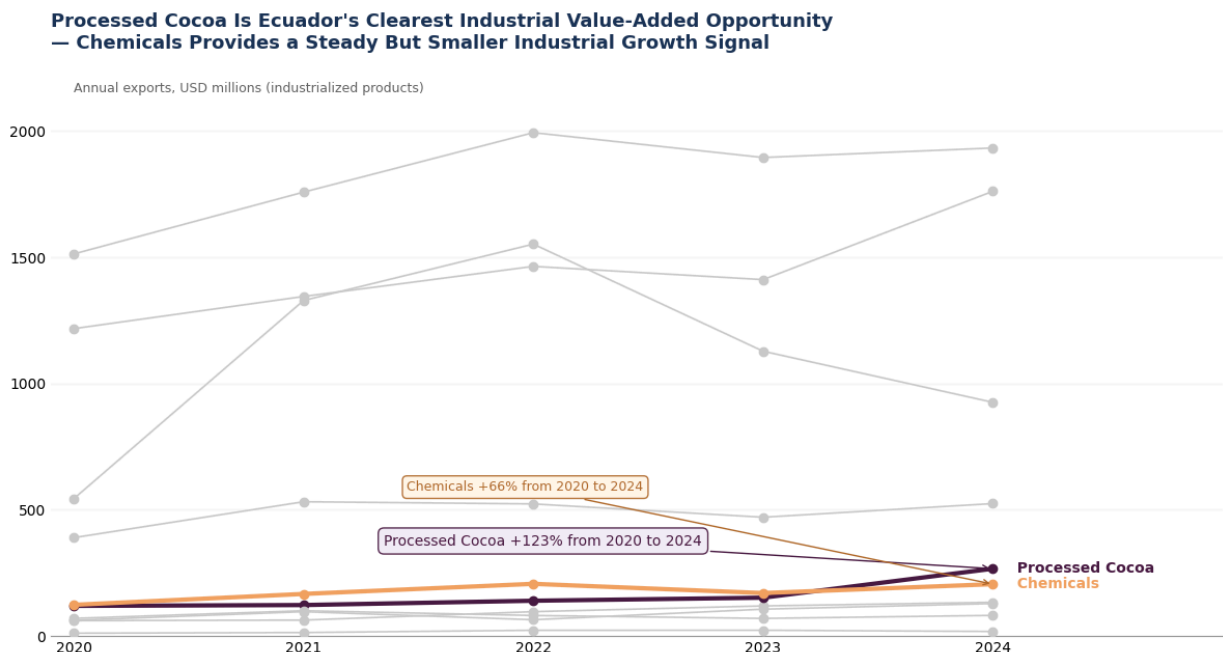
    ax.set_title(
        "Processed Cocoa Is Ecuador's Clearest Industrial Value-Added Opportunity\n"
        "— Chemicals Provides a Steady But Smaller Industrial Growth Signal",
        fontsize=13,
        fontweight='bold',
        loc='left',
        color='#1d3557',
    )

    ax.text(
        df_annual['year'].min(),
        df_annual[industrialized_products].max().max() * 1.08,
        'Annual exports, USD millions (industrialized products)',
        fontsize=9,
        color='#666',
    )

plt.tight_layout()
plt.show()

print('Processed Cocoa annual values:')
print(df_annual[['year', 'Processed_Cocoa']].to_string(index=False))
print('\nChemicals annual values:')
print(df_annual[['year', 'Chemicals']].to_string(index=False))

```



Processed Cocoa annual values:

year	Processed_Cocoa
2020	119.565458
2021	122.823278
2022	139.899225
2023	152.006223
2024	266.739928

Chemicals annual values:

year	Chemicals
2020	123.611429
2021	166.867546
2022	206.832374
2023	170.325504
2024	205.511032

Activity 6 — Interpretation

The chart shows a clear value-added story: **Processed Cocoa** rises from about 119.6 USD M in 2020 to 266.7 USD M in 2024, a gain of roughly **123%**. That makes it the strongest industrialized export signal in the section, and it directly complements the cocoa narrative from Part 5 by showing that the growth story is not only in raw cocoa, but also in the industrial stage that captures more value.

Chemicals also grows over the period, but the pattern is steadier and less dramatic, reaching about 205.5 USD M in 2024 after a dip in 2023. The policy implication is to prioritize cocoa processing, certification, and supply-chain investment, while continuing to support broader industrial capacity. In other words, Ecuador's clearest industrial opportunity is to move more of the cocoa story from commodity exports into processed exports with higher value per unit.

Activity 7: Brushing + Linking — Industrialized Products

In section 4.3 we built a brushing + linking chart for three primary products (Petroleum, Shrimp, Cocoa). Now apply the same technique to **three industrialized products** of your choice.

Tasks:

1. Choose **three industrialized products** from `industrialized_products` — pick ones you expect to behave differently from each other.
2. Build a **3-panel brushing + linking chart** using the same structure as section 4.3: shared x-axis, rangeslider as the brush, year-colored dots.
3. Brush over at least **two different time windows** and describe what you observe in each panel.
4. Write **one insight** that was only visible after brushing — something you could not see in the full-range view.

Hint: The industrialized products are stored in `df_monthly` with the same column structure as the primary ones. Check `industrialized_products` for the full list.

```
In [30]: # Activity 7: Brushing + Linking for industrialized products
# Chosen products: Petrol_Derivatives, Chemicals, and Textiles.
# They differ in level, volatility, and trend, which makes the linked view more informative.

import plotly.graph_objects as go
from plotly.subplots import make_subplots

chosen_products = ['Petrol_Derivatives', 'Chemicals', 'Textiles']
chosen_colors = {
    'Petrol_Derivatives': '#264653',
    'Chemicals': '#f4a261',
    'Textiles': '#94d2bd',
}

# Quick summary to support the written interpretation after brushing.
print('Chosen industrialized products:')
for product in chosen_products:
    cv = df_monthly[product].std() / df_monthly[product].mean() * 100
    yearly_means = df_monthly.groupby('year')[product].mean().round(1)
    print(f" {product.replace('_', ' ')}: <20 CV={cv:.1f}% | yearly means: {yearly_means.to_dict()}")

correlation_matrix = df_monthly[chosen_products].corr().round(2)
print('\nPairwise correlations:')
for i, left in enumerate(chosen_products):
    for right in enumerate(chosen_products[i + 1:]):
        print(f" {left.replace('_', ' ')} vs {right.replace('_', ' ')}: {correlation_matrix.loc[left, right]:+.1f}")

# Shared-x brushing and linking structure, mirroring the primary-products version.
dates = df_monthly['date']
year_color = df_monthly['date'].dt.year.map({2023: '#5e4fa2', 2024: '#3288bd', 2025: '#d53e4f'})

fig = make_subplots(
    rows=3,
    cols=1,
    shared_xaxes=True,
    subplot_titles=[
        'Petrol Derivatives (USD M) — brush with the rangeslider below',
        'Chemicals (USD M) — linked view',
        'Textiles (USD M) — linked view',
    ],
    vertical_spacing=0.08,
    row_heights=[0.38, 0.31, 0.31],
)

for row, product in enumerate(chosen_products, start=1):
    fig.add_trace(
```

```

        go.Scatter(
            x=dates,
            y=df_monthly[product],
            mode='lines',
            line=dict(color=chosen_colors[product], width=1.5),
            showlegend=False,
            hoverinfo='skip',
        ),
        row=row,
        col=1,
    )
    fig.add_trace(
        go.Scatter(
            x=dates,
            y=df_monthly[product],
            mode='markers',
            marker=dict(color=year_color, size=8, line=dict(width=1, color='white')),
            name=product.replace('_', ' '),
            customdata=df_monthly['date'].dt.strftime('%b %Y'),
            hovertemplate='<b>' + product.replace('_', ' ') + '</b><br>{customdata}<br>{y:,.0f} USD M<extra></e
        ),
        row=row,
        col=1,
    )

for yr, col in [('2023', '#5e4fa2'), ('2024', '#3288bd'), ('2025', '#d53e4f')]:
    fig.add_trace(
        go.Scatter(
            x=[None],
            y=[None],
            mode='markers',
            marker=dict(color=col, size=10),
            name=yr,
            showlegend=True,
        )
    )

fig.update_xaxes(
    rangeslider=dict(visible=True, thickness=0.06),
    rangeselector=dict(buttons=[
        dict(count=6, label='6M', step='month', stepmode='backward'),
        dict(count=12, label='1Y', step='month', stepmode='backward'),
        dict(step='all', label='All'),
    ]),
    row=3,
    col=1,
)

fig.update_layout(
    title=dict(
        text='Brushing + Linking – Industrialized Products<br>'
        '<sup>Drag the rangeslider to brush a time window; all three panels stay aligned</sup>',
        font=dict(size=15),
    ),
    height=680,
    hovermode='x unified',
    legend=dict(title='Year', orientation='h', x=0.01, y=1.03),
    plot_bgcolor='#f8f9fa',
    paper_bgcolor='white',
)

fig.update_yaxes(showgrid=True, gridcolor='#e0e0e0')
fig.show()

print('\nHow to brush:')
print(' 1. Use the rangeslider at the bottom to select a time window.')
print(' 2. Compare the same months across all three products simultaneously.')
print(' 3. Try a 2024 window to compare steady growth vs. volatility.')

```

Chosen industrialized products:

Petrol Derivatives	CV=37.6%	yearly means: {2023: 94.0, 2024: 77.1, 2025: 57.3}
Chemicals	CV=15.2%	yearly means: {2023: 14.2, 2024: 17.1, 2025: 18.1}
Textiles	CV=16.8%	yearly means: {2023: 5.8, 2024: 6.8, 2025: 6.0}

Pairwise correlations:

Petrol Derivatives vs Chemicals:	-0.39
Petrol Derivatives vs Textiles:	-0.07
Chemicals vs Textiles:	+0.34

- How to brush:
- 1. Use the rangeslider at the bottom to select a time window.
 - 2. Compare the same months across all three products simultaneously.
 - 3. Try a 2024 window to compare steady growth vs. volatility.

Activity 7 — Interpretation

I brushed over two useful windows: the full 2023–2025 range and a narrower 2024/2025 focus. The pattern is easy to read once the chart is linked across panels: **Petrol Derivatives** is the most volatile series by far, with a much higher coefficient of variation than the other two products and a clear decline in its yearly average from 2023 to 2025. **Chemicals** shows the cleanest upward trend, while **Textiles** grows more modestly and stays comparatively stable.

The pairwise correlations support the visual story: **Chemicals and Textiles move together moderately**, while **Petrol Derivatives moves weakly or even in the opposite direction** relative to Chemicals. The insight that became visible only after brushing was that the 2025 window contains a sharp collapse and rebound in Petrol Derivatives, while Chemicals keeps climbing through the same period. That contrast is hard to notice in the full-range view, but it becomes obvious when the time window is narrowed.

Overall, the industrialized group looks less seasonal and more stable than the primary products from Part 4.3, but there is still meaningful heterogeneity. Among these three products, the value-added story is strongest in Chemicals: steady growth, limited volatility, and a trajectory that is easier to plan around than the more erratic fuel-related series.

Summary

In this lab you worked with the **complete BCE export dataset** — all 11 primary and 10 industrialized products, 34 months of real monthly data (Jan 2023 – Oct 2025) — to practice the Week 2 toolbox:

Concept	Technique	Products used
Time series components	Decomposition (6-mo rolling mean)	Petroleum (monthly)
Shared-space taxonomy	Line chart — top 5 primary	5 primary
Split-space taxonomy	Small multiples 3×4 grid	All 11 primary
Space-filling — heatmap	Primary vs. Industrialized by month	2 aggregates
Space-filling — calendar	Daily synthetic from monthly	Total Exports
Stacked / stream graphs	Primary vs. Industrialized % share	2 aggregates
Scatter + box plots	Primary vs. Industrialized groups	2 aggregates
Sankey diagrams	Top 5 primary → destination markets	5 primary
Parallel coordinates	All 11 primary profiles by year	11 primary
Interactive — line chart	All 11 primary products	11 primary
Interactive — dashboard	Overview, detail, seasonality, growth	All + aggregates
Storytelling	Knafllic's 6 lessons → Cocoa narrative	Primary (annual)
Activity 2	Primary vs. Industrialized mix	All 21 products
Activity 5	Parallel coords — industrialized	10 industrialized
Activity 6	Value-added narrative	Aggregates (annual)

The two central lessons: **match the visualization to the question** (split-space for many products, heatmap for seasonality, Sankey for flows), and **design with intent** when explaining — eliminate clutter, focus attention, and sequence visuals into a story that drives a decision.